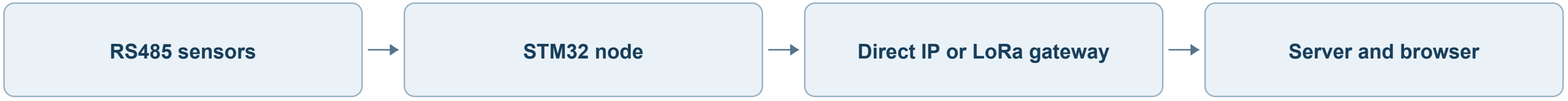
WIOT software flowcharts

A complete functional map of the sensor node, LoRa gateway, Python server and test tools.

This reference follows the active Arduino STM32 release, including four-sensor acquisition, RTC consumption, modem recovery, valve control, power policies and the external telemetry log. The alternate ESP32 and native CubeIDE ports have a separate boundary in F30.



How to use the reference

Start with the system	F01 whole system, F02 configuration, F03 startup and F04 scheduler.
Follow one feature	F05-F10 sensing and payloads; F11-F18 communications, commands and recovery.
Understand sleep and storage	F19-F25 cover supported power combinations, wake behavior and flash history.
Trace the server and tools	F26-F30 cover ingestion, access, simulators, release checks and alternate ports.

Read each arrow in its indicated direction. Dashed arrows return to an earlier phase or cross an already placed peer; they do not mean a background thread. Process blocks may summarize a bounded function or state-machine phase.

The PDF uses A3 landscape for legible engineering charts. The HTML viewer is offline, searchable and zoomable. Editable DOT and Mermaid sources accompany every diagram.

Reading a software flowchart does not prove hardware qualification. No private credentials or AES key values are included.

Open chart index >

Chart index

Choose an ID to jump to the full chart. The command catalog and power combination matrix follow the thirty flowcharts.

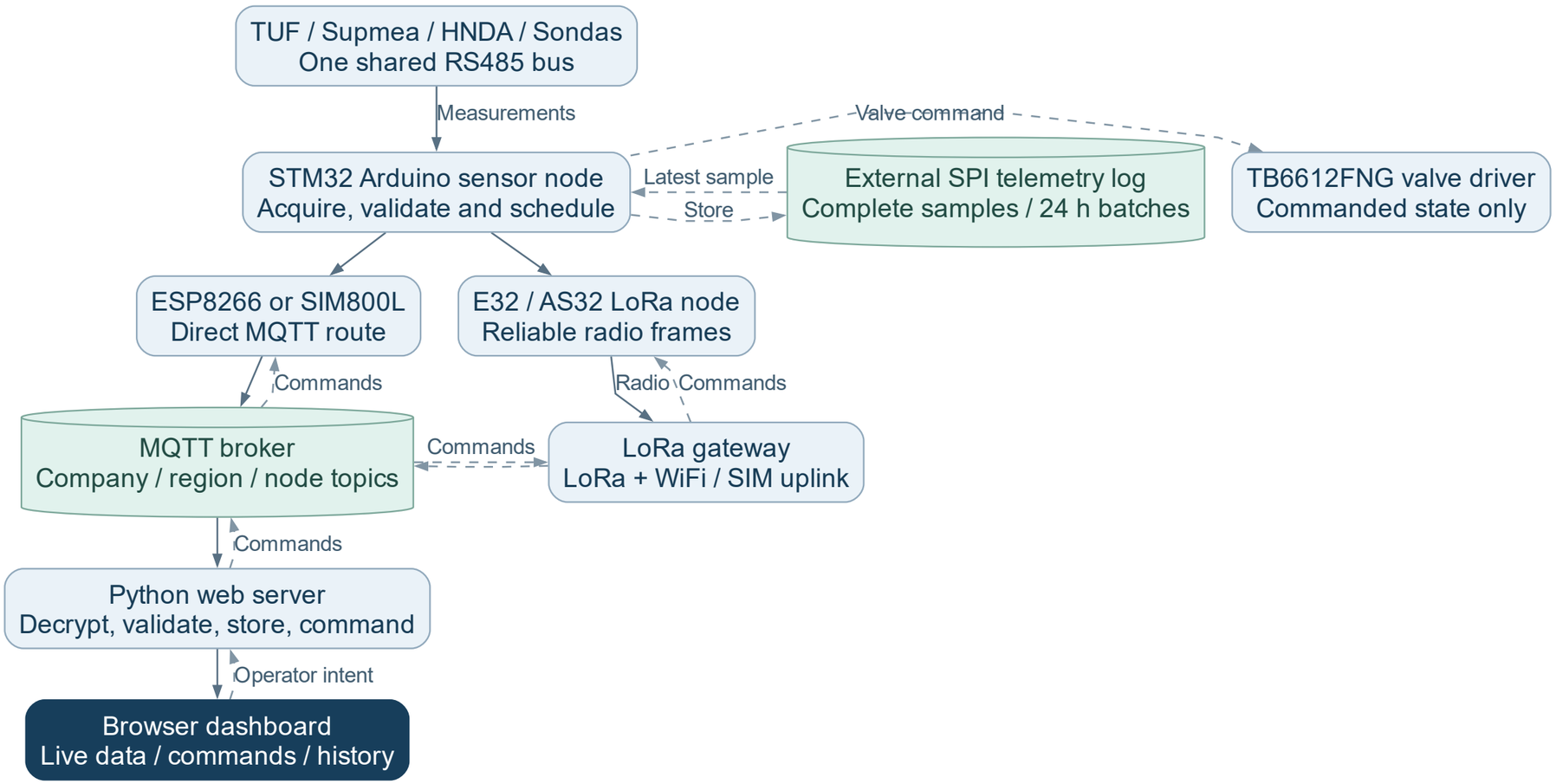
F01	Whole system data and control paths	3
F02	Choose the compiled product	4
F03	Boot and reset initialization	5
F04	Sensor node cooperative main loop	6
F05	RS485 request response and failure path	7
F06	TUF flow and native period totals	8
F07	Supmea live data and cumulative totals	9
F08	HNDA and Sondas soil measurements	10
F09	Supmea RTC day month and year consumption	11
F10	Build a truthful telemetry payload	12
F11	Direct MQTT receive and send priorities	13
F12	WiFi SIM connection and retry recovery	14
F13	LoRa wire transport and reassembly	15
F14	Gateway bridge and downlink forwarding	16
F15	Server command intent and delivery lifecycle	17
F16	Node command dispatch replay and replies	18
F17	Commissioned latching valve pulse	19
F18	Watchdog reset causes and two counters	20
F19	Select a supported power policy	21
F20	Remote wake with normal connected modem	22
F21	Periodic and queued RTC Standby	23
F22	LoRa wake on radio with STM32 Stop	24
F23	Capture and commit a flash sample	25
F24	Flash recovery retention and clearing	26
F25	Download retained history with persistence ACK	27
F26	Server receive validate and store	28
F27	Server users dashboard reports and isolation	29
F28	Four sensor simulator on one shared bus	30
F29	Verify document and release the project	31
F30	Active release and separate firmware ports	32

A1 Command catalog (IDs 1-14)

A2 Supported power combinations

Whole system data and control paths

One sensing architecture supports direct IP nodes and LoRa nodes behind a gateway. Commands travel back through the selected route.



- The gateway forwards the encrypted node payload; the server decodes it. LoRa-only nodes do not use their IP-modem path.

- Solid arrows show functional data/control direction. All subsequent charts describe software behavior, not a new hardware wiring diagram.

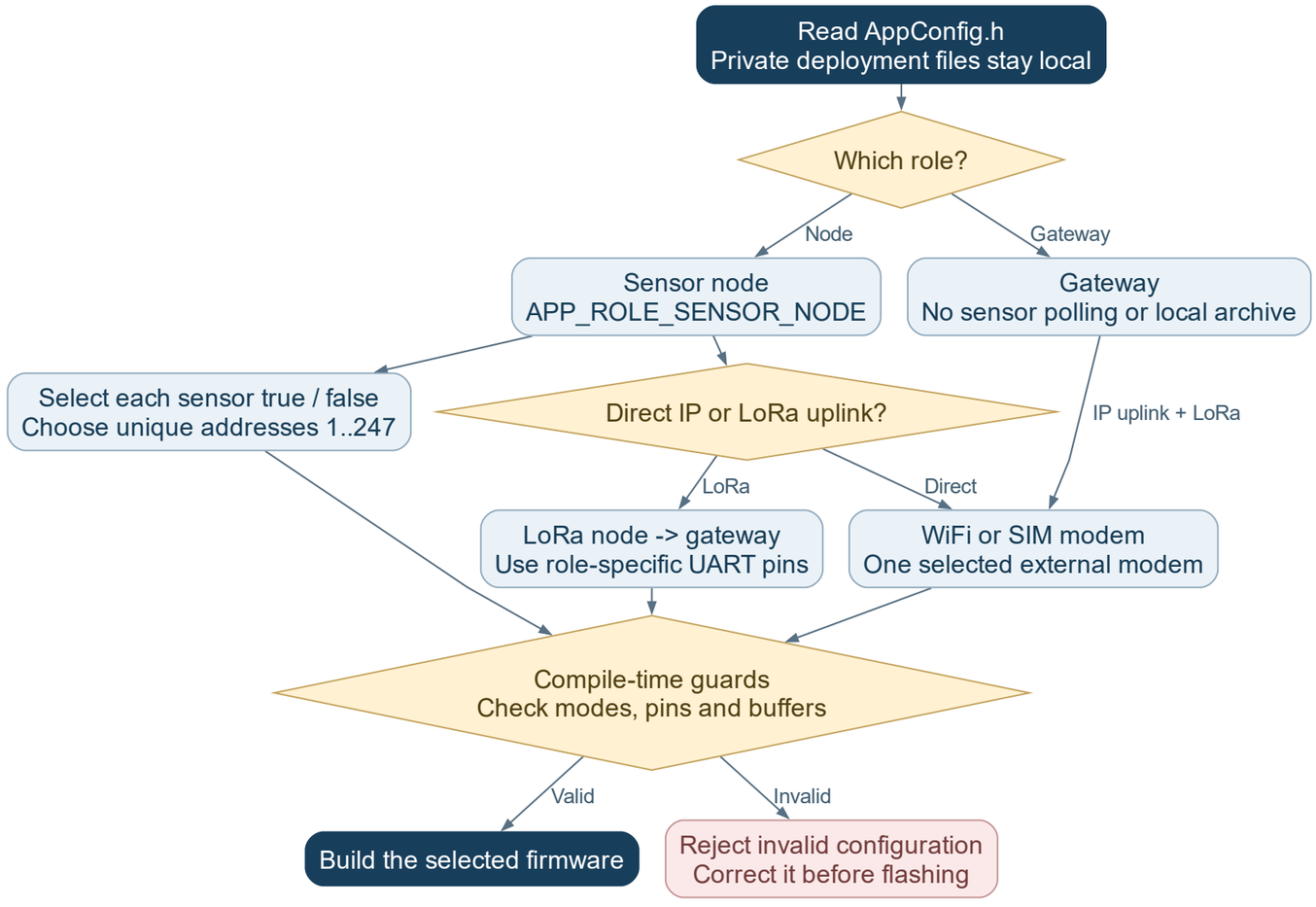
Source locations

Node/IoT_Node_Encrypted.ino:196
Node/src/app/GatewayApp.cpp:58
Server/server_web.py:987

Index >

Choose the compiled product

Build-time macros decide the role, active transport, sensor requests, addresses and power policy before any hardware is used.



- All four sensors can be enabled together. There is no longer a mandatory exclusive two-sensor project selection.

- Public defaults select node 2, Company_1 / Region_1, LoRa WOR with UART RX wake, all four sensors, SPI logging and LITE debug. The 4 November bench copy selects node 3, disables all four sensor polls and uses FULL debug. Verify the header for each flashed image.

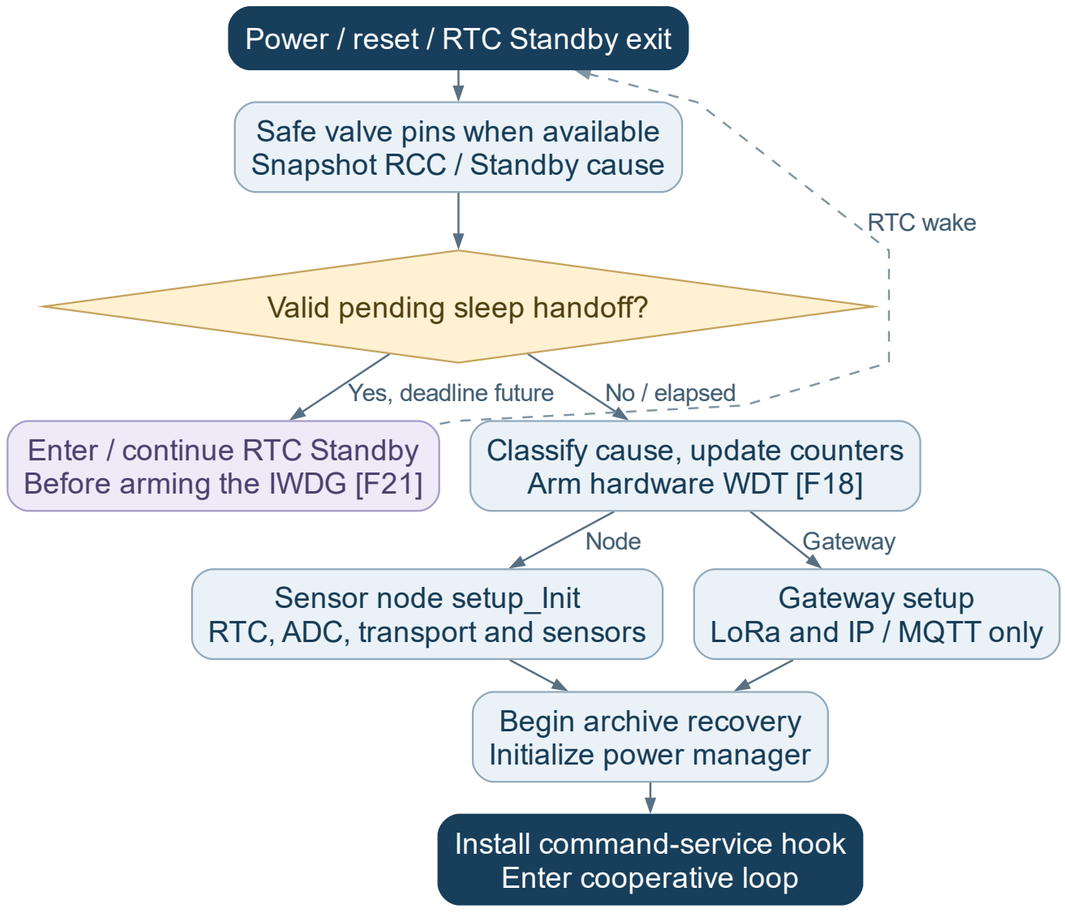
Source locations

Node/src/config/AppConfig.h:218
Node/src/config/AppConfig.h:464

Index >

Boot and reset initialization

Safety and the Standby handoff run before the IWDG; normal active initialization then selects the node or gateway path.



- The default RTC date is not proof of trusted time. STIME establishes validated UTC time for calendar totals and flash retention.

- The IWDG protects active startup calls as well as loop execution. A valid long Standby handoff deliberately occurs before it starts.

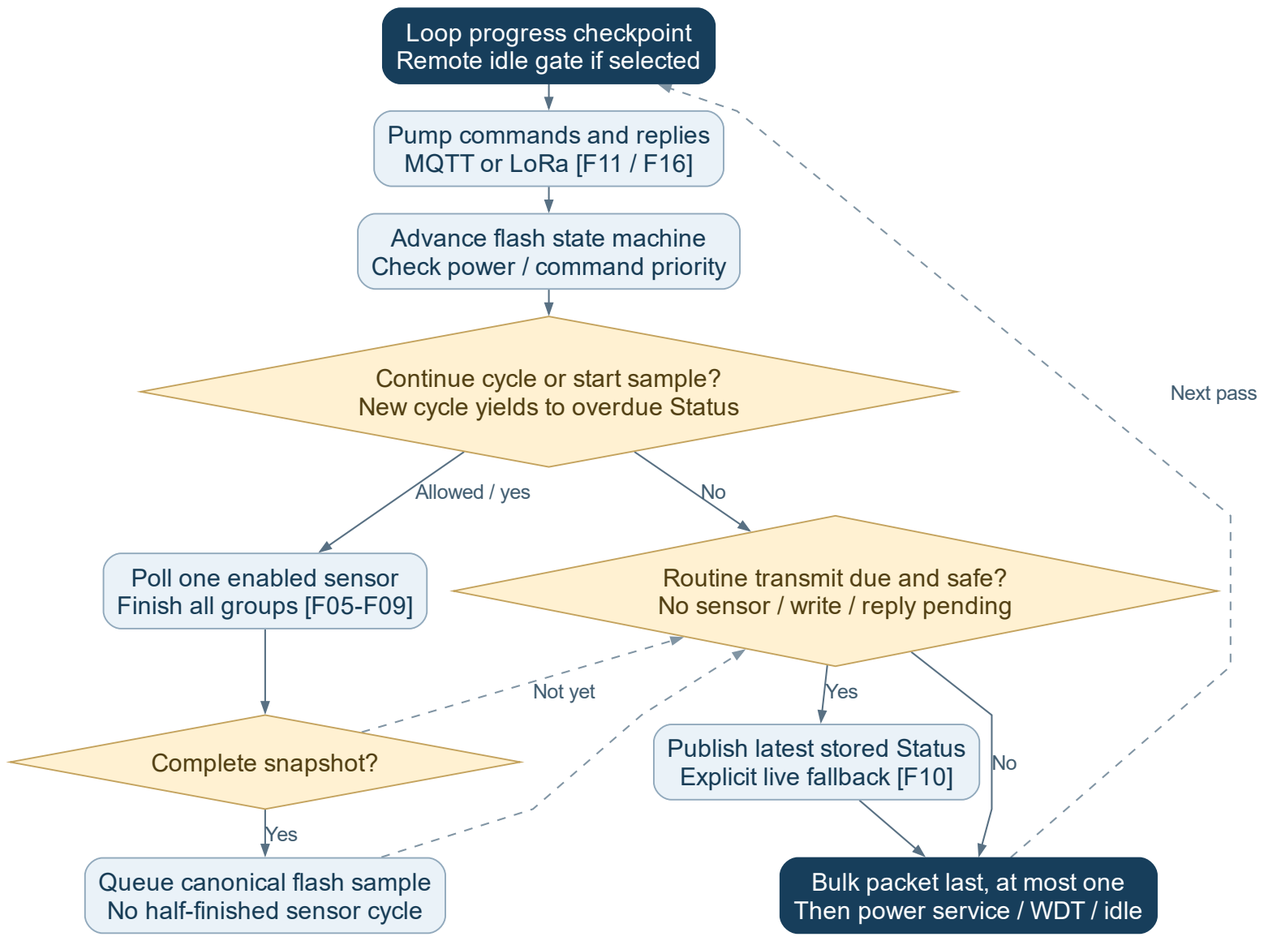
Source locations

Node/IoT_Node_Encrypted.ino:196
Node/src/app/IoT_Node.cpp:453
Node/src/power/PowerManager.cpp:858

Index >

Sensor node cooperative main loop

Commands and replies precede sampling, routine uploads and bulk history. One enabled sensor is serviced per application pass.



- Priority is cooperative: an in-flight modem, sensor or logical LoRa send cannot be preempted instantly.

- NORMAL, scheduled REMOTE_WAKE and default LORA_WOR use independent SRATE and STRAN deadlines. Legacy WOR and timed Standby retain combined-cycle policies [F20-F22].

- SensorPollCycle finishes an active cycle, then gives an overdue Status an opportunity before another cycle starts. Failed or disabled sensors can still produce zero/invalid samples and reports. Pending command or flash write work can defer the report.

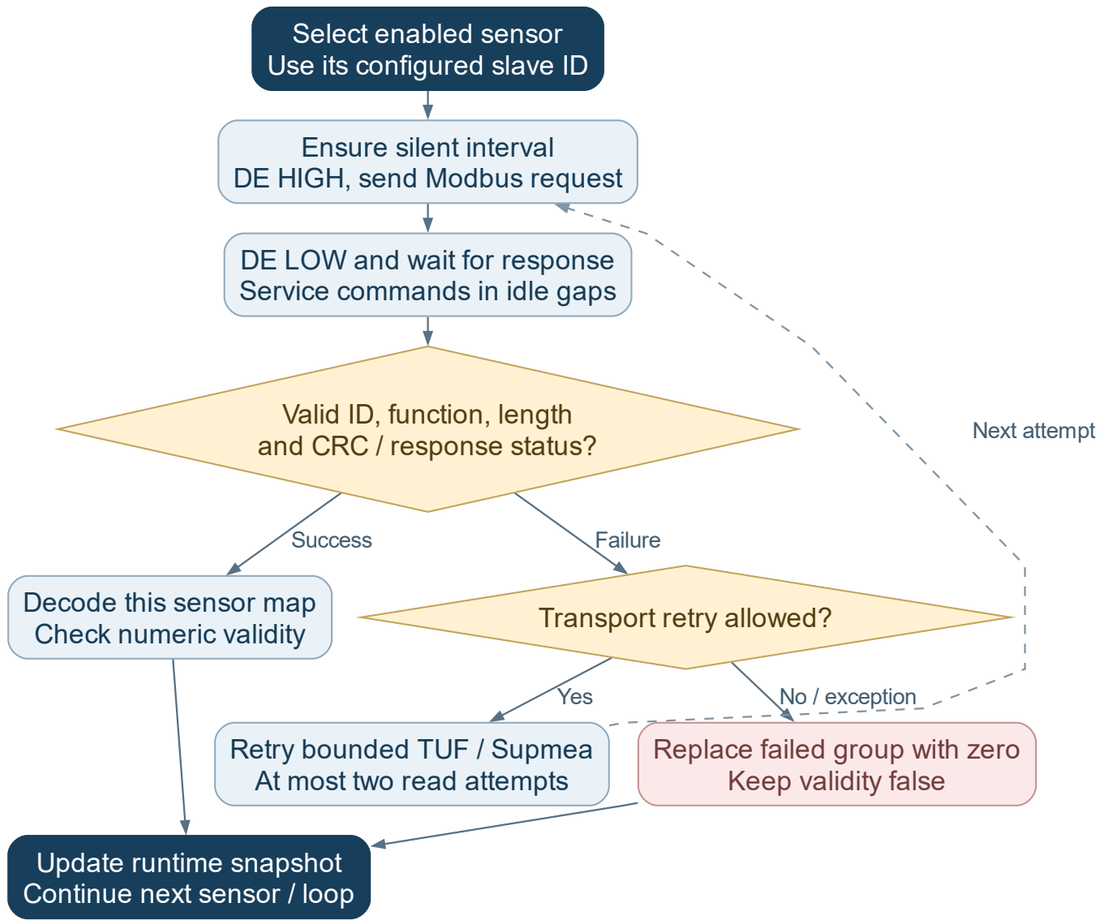
Source locations

Node/IoT_Node_Encrypted.ino:229
Node/src/app/SensorPollCycle.h:7

Index >

RS485 request response and failure path

All four drivers share one half-duplex bus. Each request is bounded and every returned group carries its own validity.



- Bus setting is 9600 baud, 8N1. TUF/Supmea retry transport errors; a valid Modbus exception is not retried. HNDA/Sondas reads use their current single-call path.

- The command idle hook pauses watchdog feeding so an endless Modbus wait cannot keep the controller alive forever.

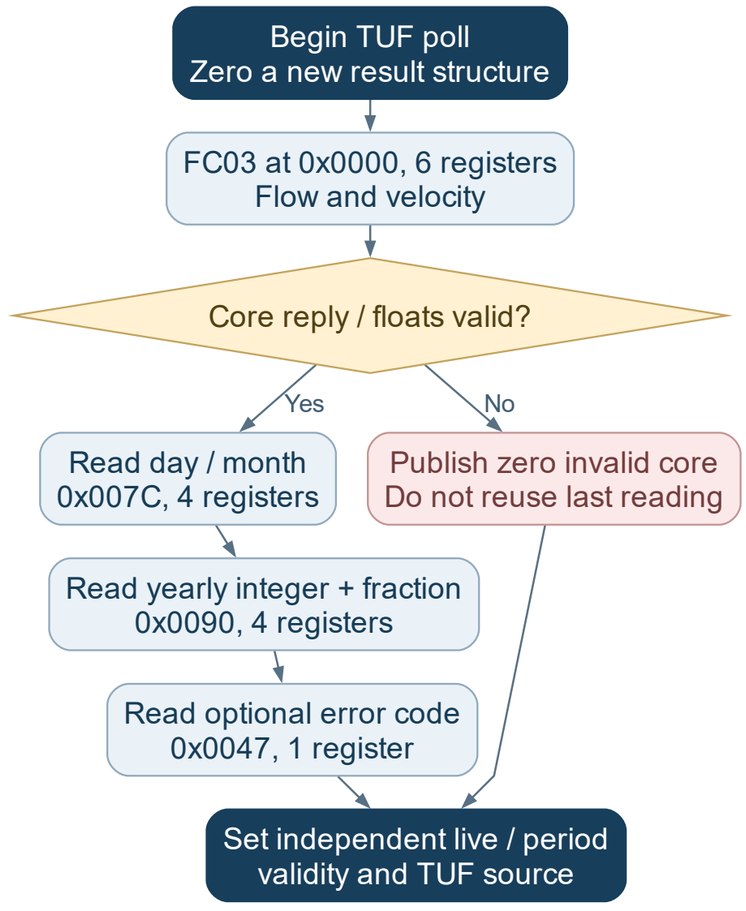
Source locations

Node/src/devices/Rs485Bus.cpp:38
Node/src/devices/Rs485Bus.cpp:28
Node/src/devices/TufSensor.cpp:60

Index >

TUF flow and native period totals

TUF supplies its own day/month/year volumes. Its 32-bit values use low-word-first register pairs, unlike Supmea.



- Modbus bytes within each 16-bit register retain standard bus order. The TUF driver combines the low register word first; it does not blindly reverse every byte.

- Day/month are float values. Year is signed integer part plus float fraction. Optional totalizer failures do not invalidate an otherwise good live group.

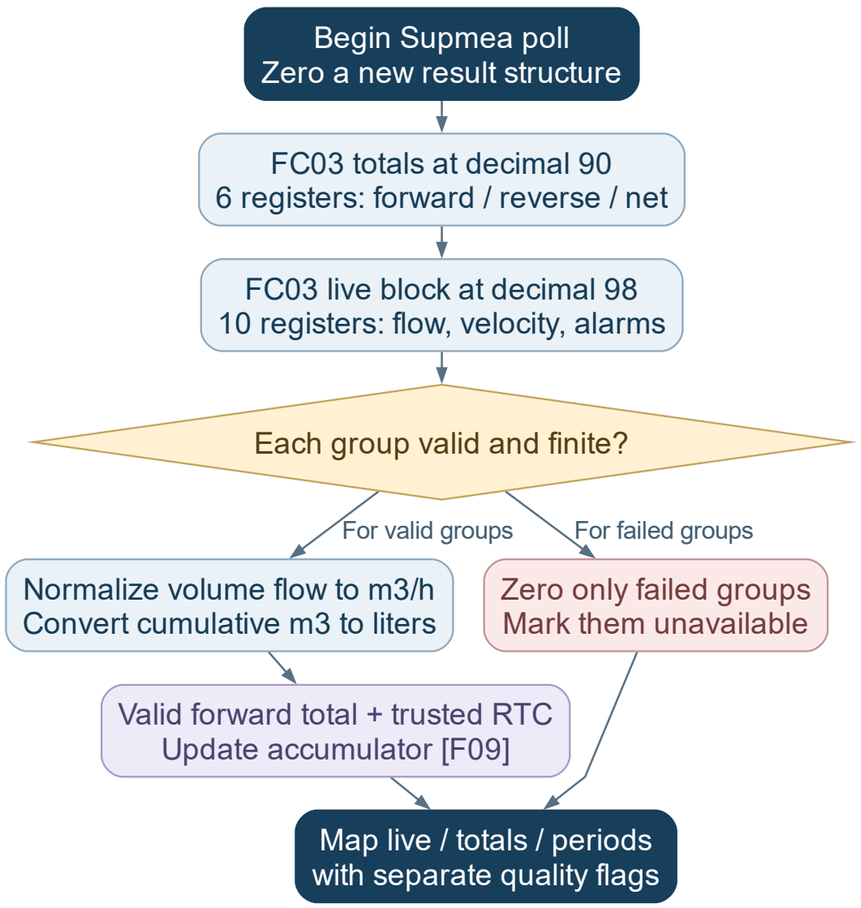
Source locations

Node/src/devices/TufSensor.cpp:128
Node/src/app/FlowMeterManager.cpp:108

Index >

Supmea live data and cumulative totals

The FMC240 driver decodes high-word-first IEEE-754 floats and normalizes volumetric flow before updating RTC consumption.



- Alarm bits include EPD and excitation-current alarm. A valid Modbus response does not by itself mean alarm-free physical measurement.

- L/h, L/min, L/s, m3/h, m3/min and m3/s are supported. Mass-flow units are not silently treated as volumetric flow.

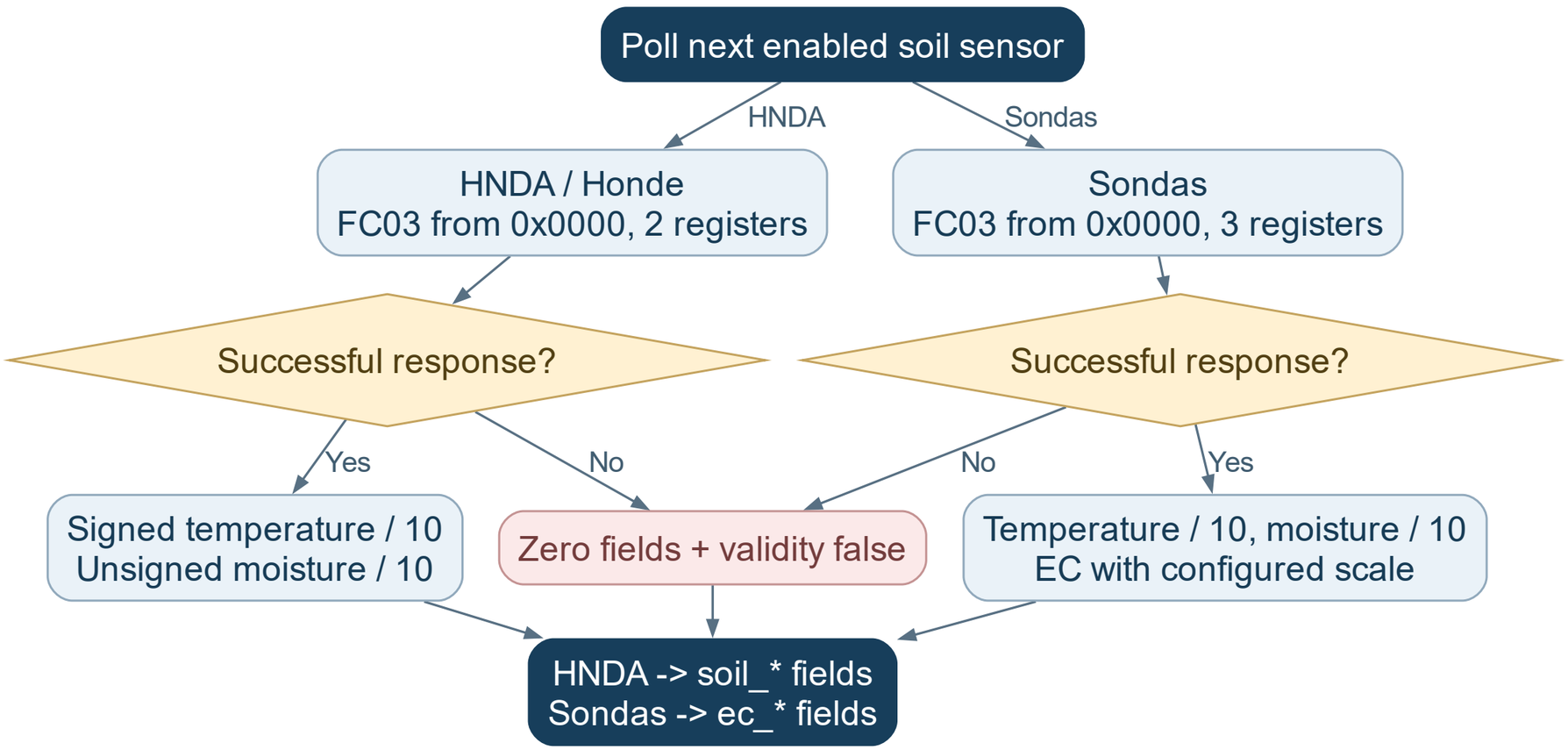
Source locations

Node/src/devices/SupmeaSensor.cpp:129
Node/src/app/FlowMeterManager.cpp:161

Index >

HNDA and Sondas soil measurements

The two soil probes have separate addresses, enable switches and payload fields even when both are on the same RS485 pair.



- HNDA is the Honde sensor called Honda in earlier project discussions. It is named HNDA in the current code.

- The Sondas register layout is configurable; the supplied family document did not establish a complete purchased-probe map. Confirm it on the actual probe.

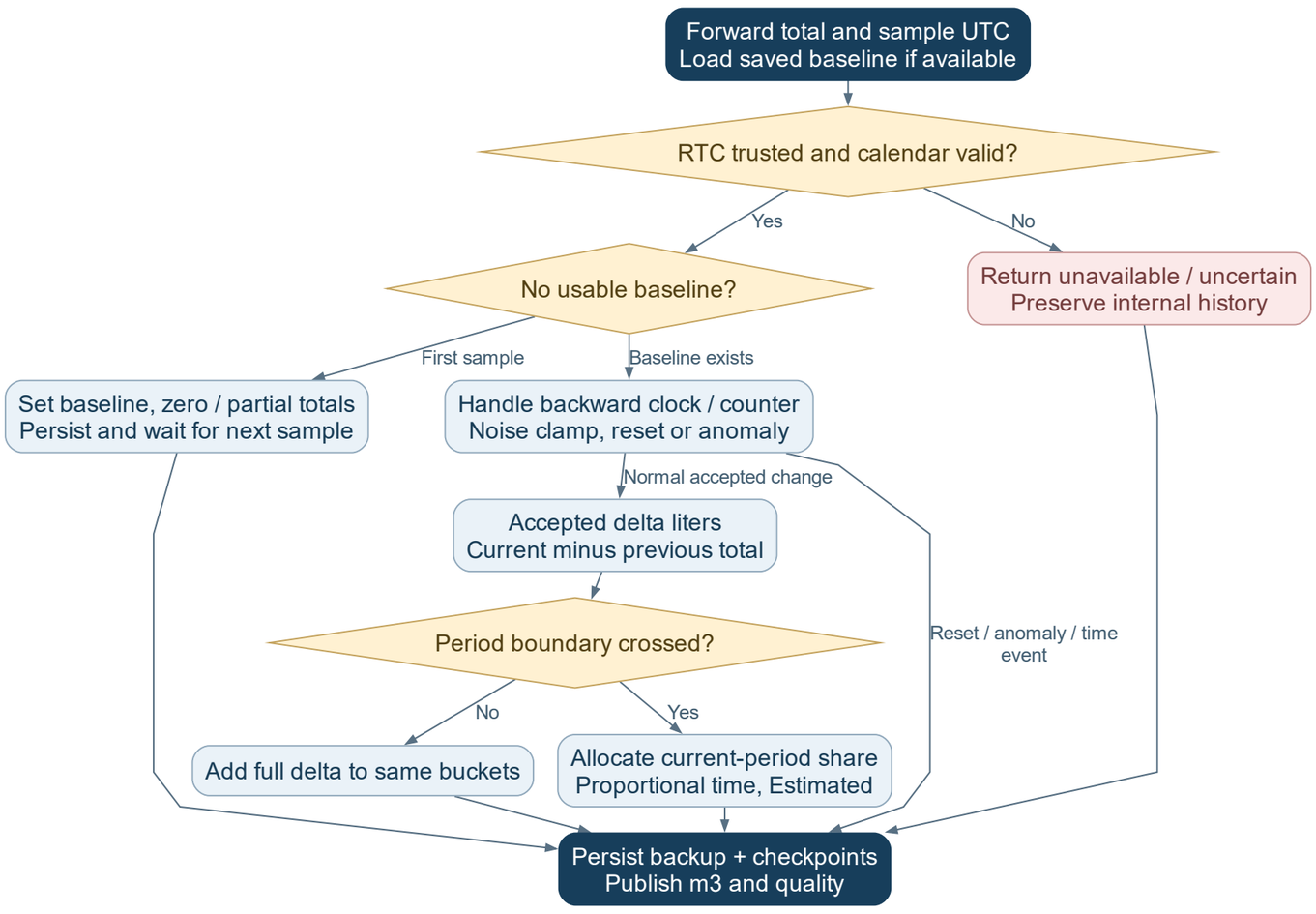
Source locations

Node/src/devices/HndaSensor.cpp:63
Node/src/devices/SondasSensor.cpp:63
Node/src/comm/SensorPayloadMapper.h:50

Index >

Supmea RTC day month and year consumption

Consumption comes from the accepted change in cumulative forward liters. RTC calendar keys decide the current day/month/year buckets.



- UTC RTC is converted using the configured UTC, fixed-offset or Africa/Cairo calendar policy. Cross-boundary allocation is an estimate, not a meter-provided daily register.

- The first partial period cannot recover water consumed before its baseline. Backup state and durable checkpoints reduce loss but do not invent missing history.

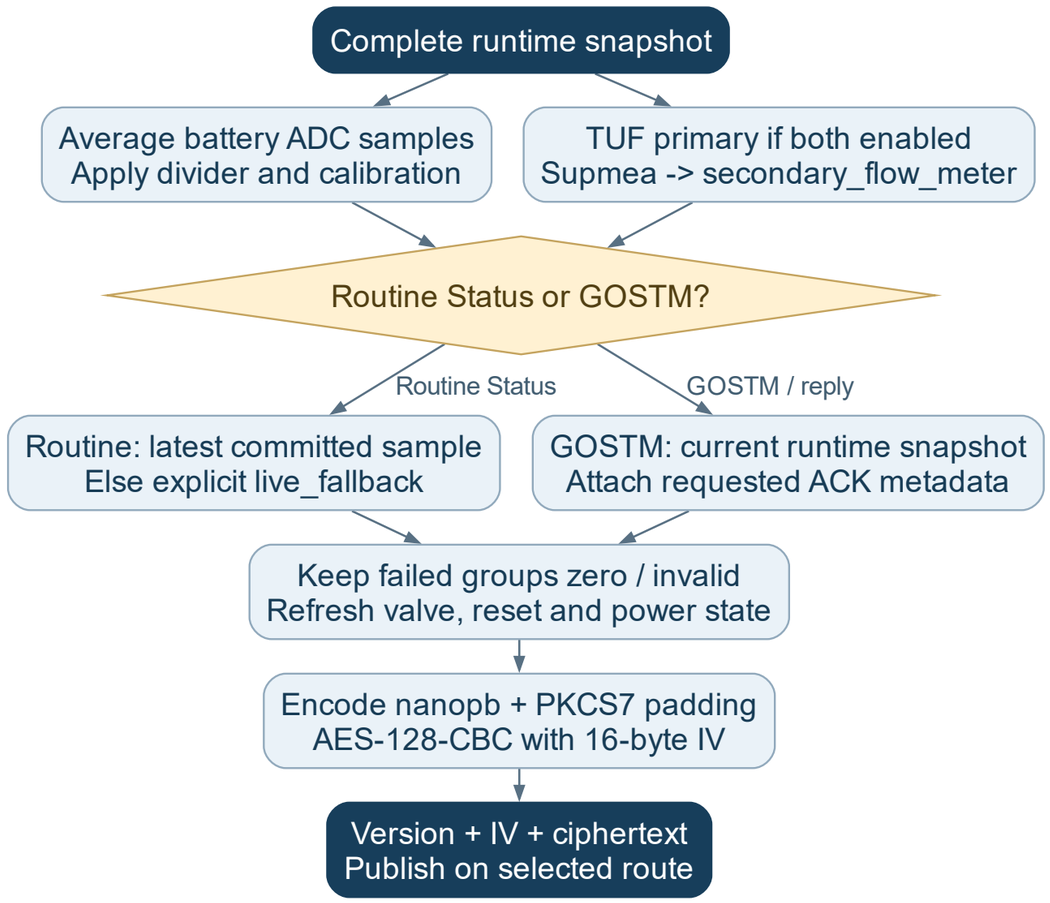
Source locations

Node/src/app/SupmeaPeriodAccumulator.cpp:503
Node/src/devices/RTC.cpp:352

Index >

Build a truthful telemetry payload

Sensor validity, acquisition time and operational state remain explicit from runtime mapping through protobuf and the existing AES envelope.



- Stored Status retains acquisition time/sequence, while commanded valve state and reset/power fields are current. HNDA and Sondas keep independent soil fields.

- The existing AES-CBC envelope has no authentication tag, and its runtime IV source is not cryptographically secure. No secret key is printed in this documentation.

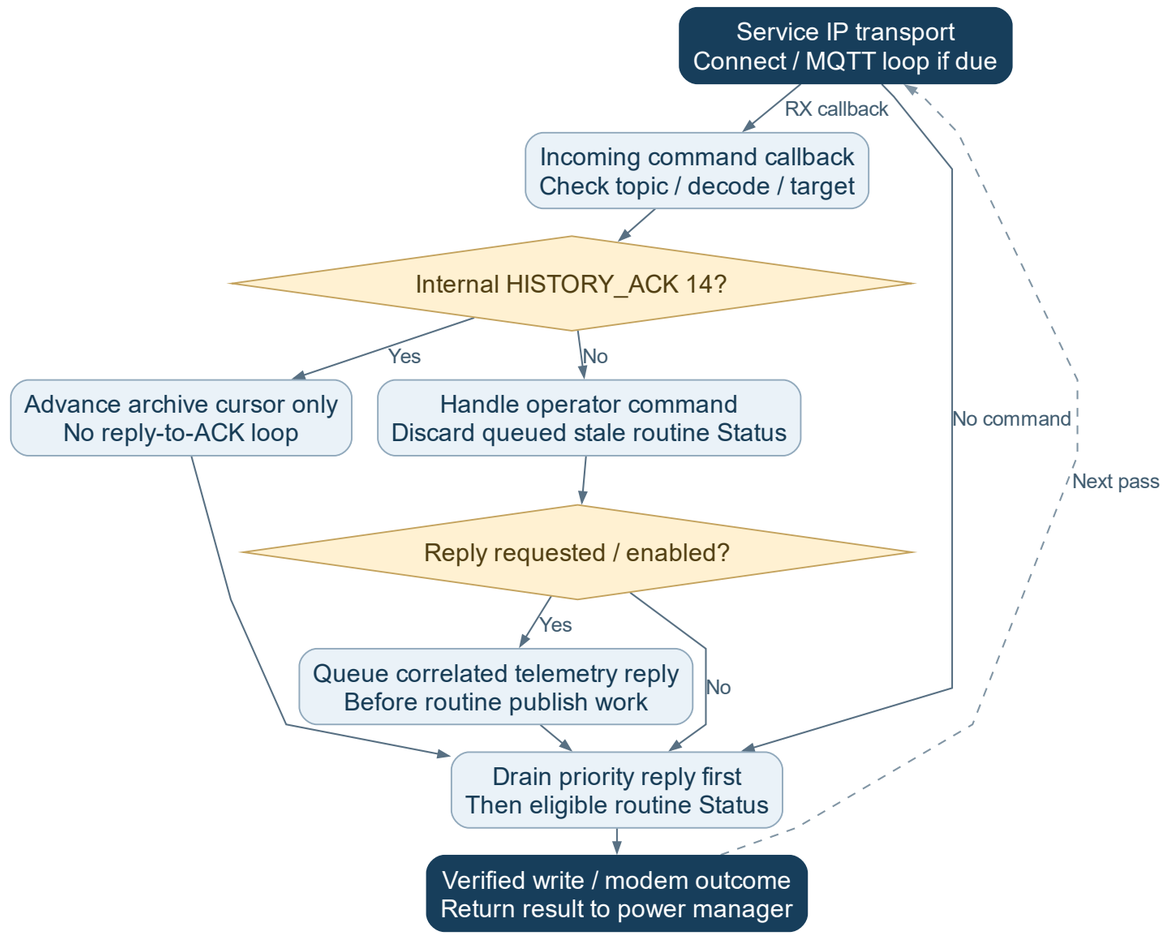
Source locations

Node/src/devices/BatteryMonitor.cpp:39
Node/src/comm/TelemetryPayload.cpp:20
Node/src/comm/PacketCodec.cpp:84

Index >

Direct MQTT receive and send priorities

TinyGSM owns AT transactions. Checked buffered sends and a bounded priority queue keep command replies ahead of routine status.



- The application queue is bounded; this is not an unlimited offline mailbox. Returned write success is checked, including partial modem writes.

- Modem SEND OK or MQTT QoS 0 acceptance is not server execution or persistence. History uses its additional application ACK [F25].

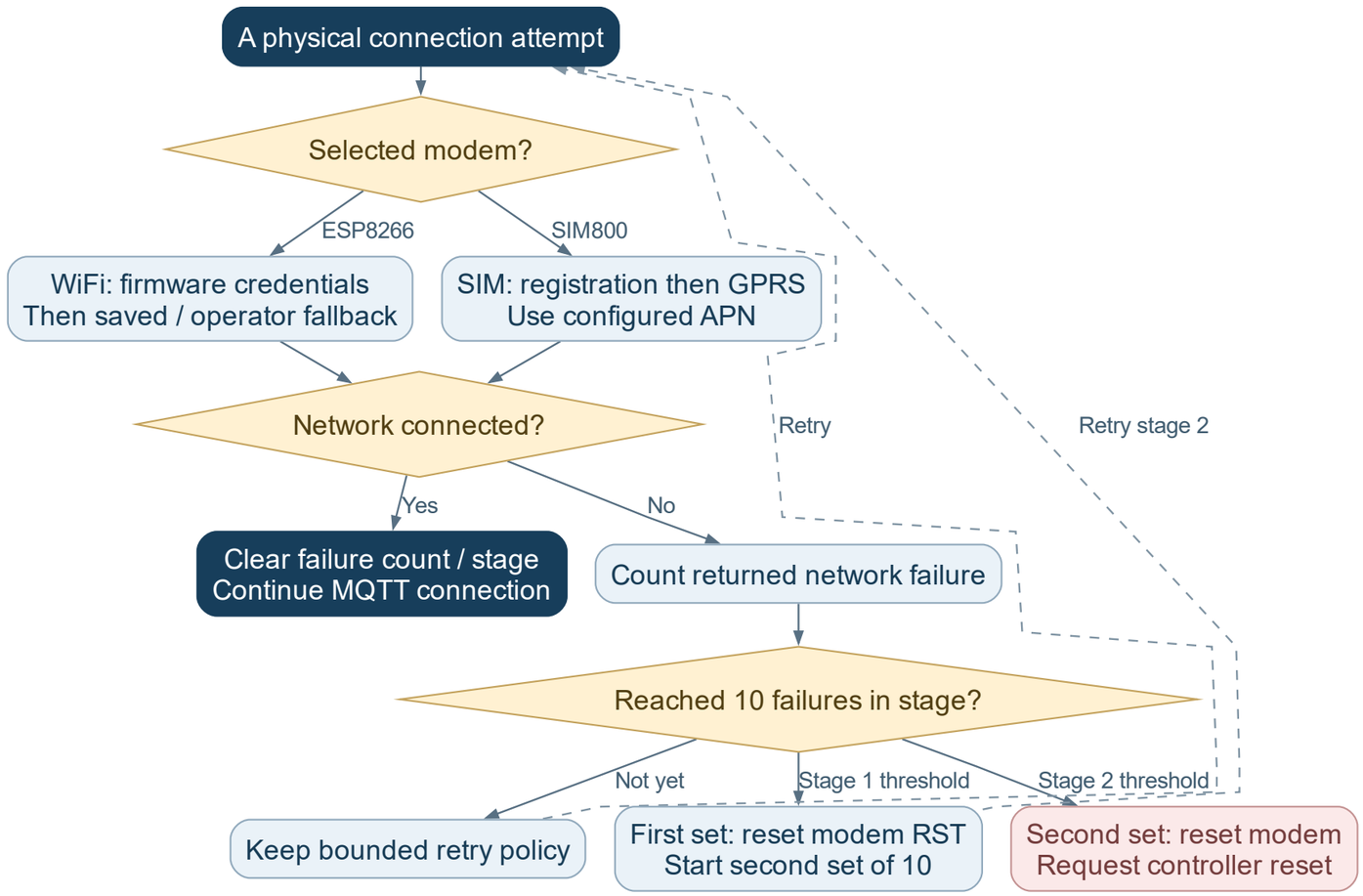
Source locations

Node/src/comm/MQTT.cpp:313
Node/src/comm/MQTT.cpp:677
Node/src/comm/PriorityPublishQueue.h:11

Index >

WiFi SIM connection and retry recovery

Physical network failures count separately from broker-only failures. Recovery has a modem-reset stage followed by a controller-reset stage.



- A call that never returns is handled by the 30 s IWDG, not the returned-failure counter. With IWDG armed, the deliberate controller reset is itself watchdog-driven.

- WiFi tries compiled settings first; saved flash credentials are fallback. A lengthy network outage can prevent sampling during startup/retries; continuous offline acquisition is not guaranteed.

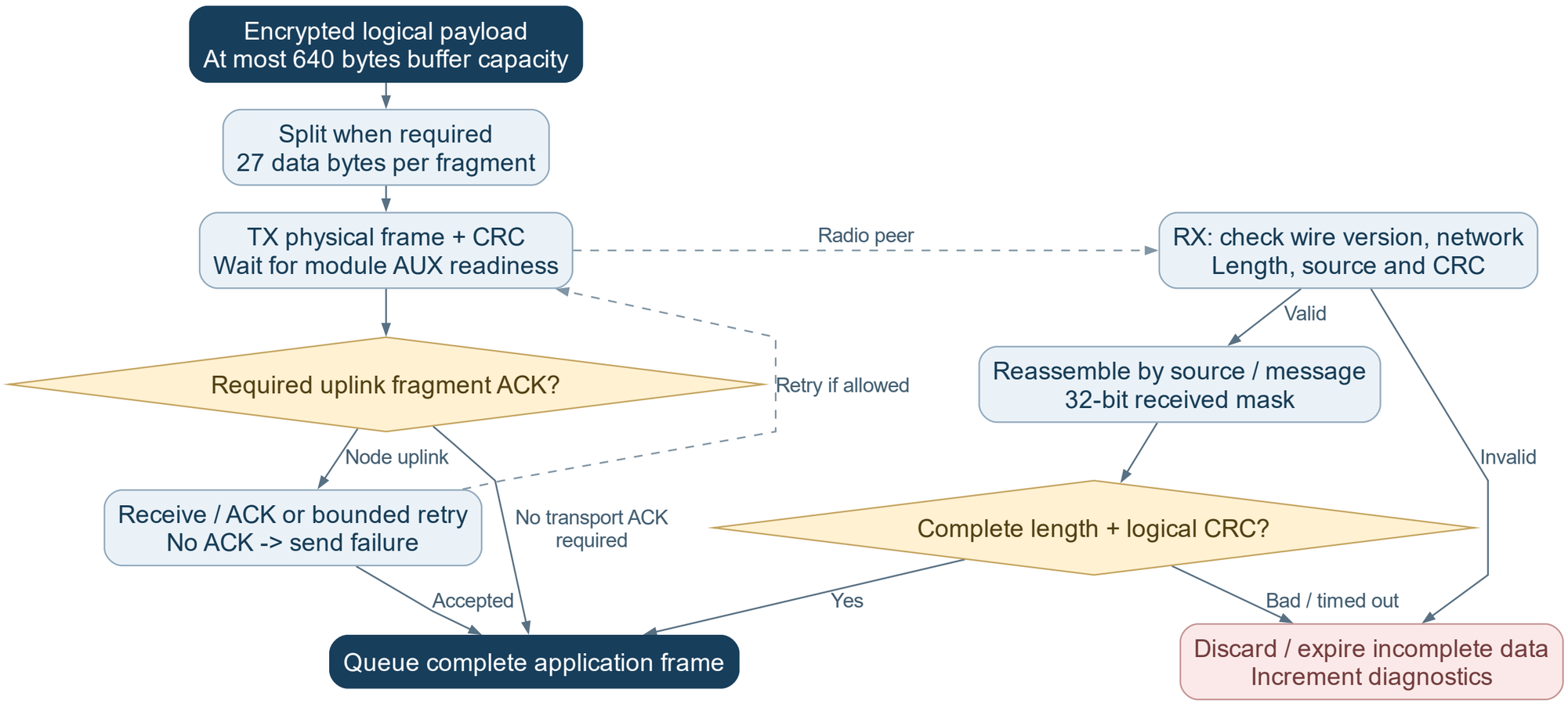
Source locations

Node/src/app/IoT_Node.cpp:342
Node/src/app/NetworkRetryPolicy.h:10
Node/src/comm/MQTT.cpp:239

Index >

LoRa wire transport and reassembly

Small physical frames carry a bounded encrypted logical message. CRC, fragment identity, retry and reassembly protect the radio transport.



- Physical wire maximum is 48 bytes; the current 640-byte capacity needs at most 24 fragments. Fragment ACKs confirm radio transport, not server record persistence.

- Use matched radio settings and unique node IDs on a shared channel. Traffic can collide. E32-TTL-100 UART RSSI is unsupported, so no invented fixed dBm is used.

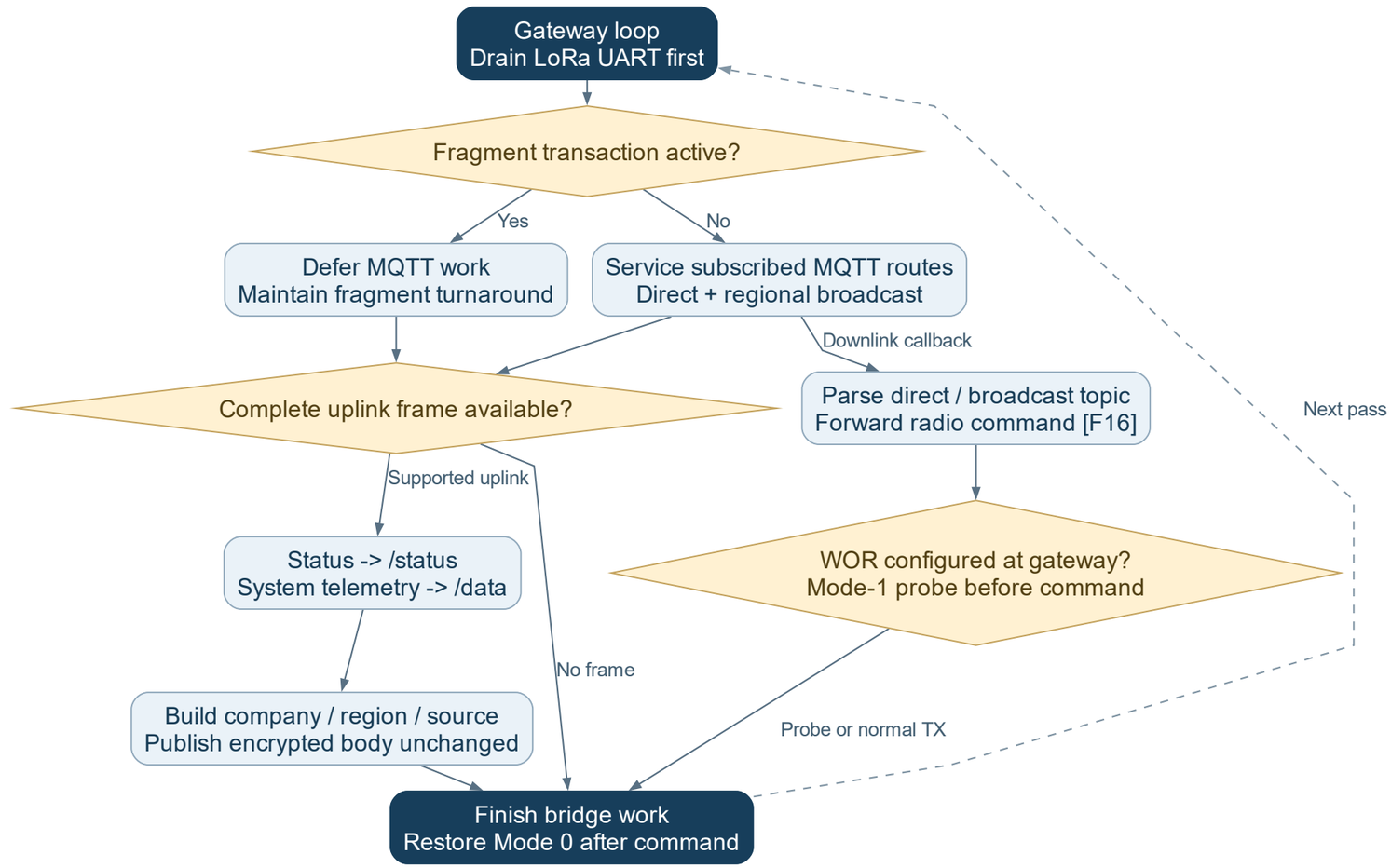
Source locations

Node/src/radio/LoraLink.cpp:1383
Node/src/radio/LoraLink.cpp:858
Node/src/radio/AS32_Config.h:81

Index >

Gateway bridge and downlink forwarding

The gateway gives an active radio fragment transaction priority, then bridges complete encrypted payloads using regional node topics.



- Gateway role remains controller NORMAL. It does not poll the sensor bus or keep the leaf node SPI telemetry log.

- The gateway subscribes to wiot/{company}/{region}/+/cmd and wiot/{company}/{region}/broadcast. A failed write for either subscription disconnects MQTT and schedules reconnect.

- Larger storage replies require an updated gateway as well as updated nodes. Server history persistence is confirmed by HISTORY_ACK.

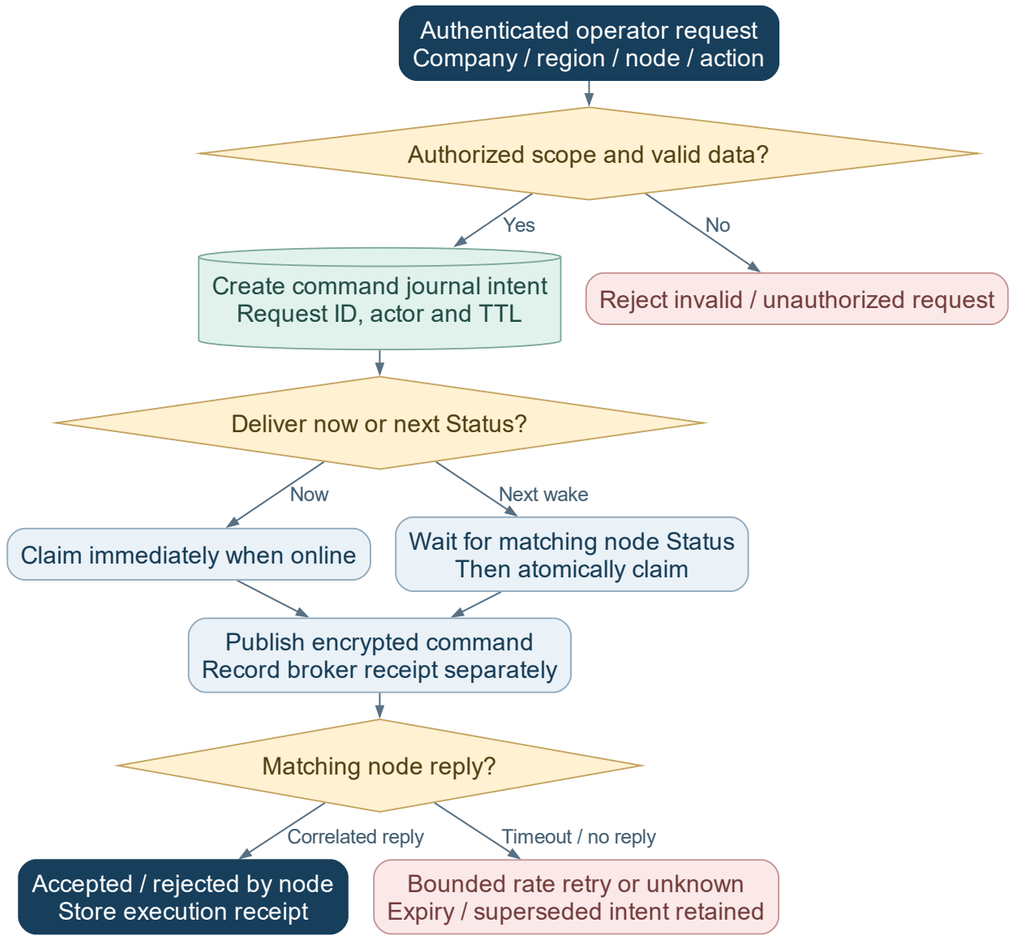
Source locations

Node/src/app/GatewayApp.cpp:58
Node/src/comm/MQTT.cpp:371

Index >

Server command intent and delivery lifecycle

Commands are durable, scoped operator intents. Broker receipt and matching node execution replies are separate events.



- Only SRATE and STRAN have bounded automatic retries. Valve, STIME, counter clears and flash commands are not automatically repeated as uncertain actions.

- Broadcast replies describe individual nodes, never confirmed execution by an entire fleet. A sleeping node must be reachable or wait until its next report.

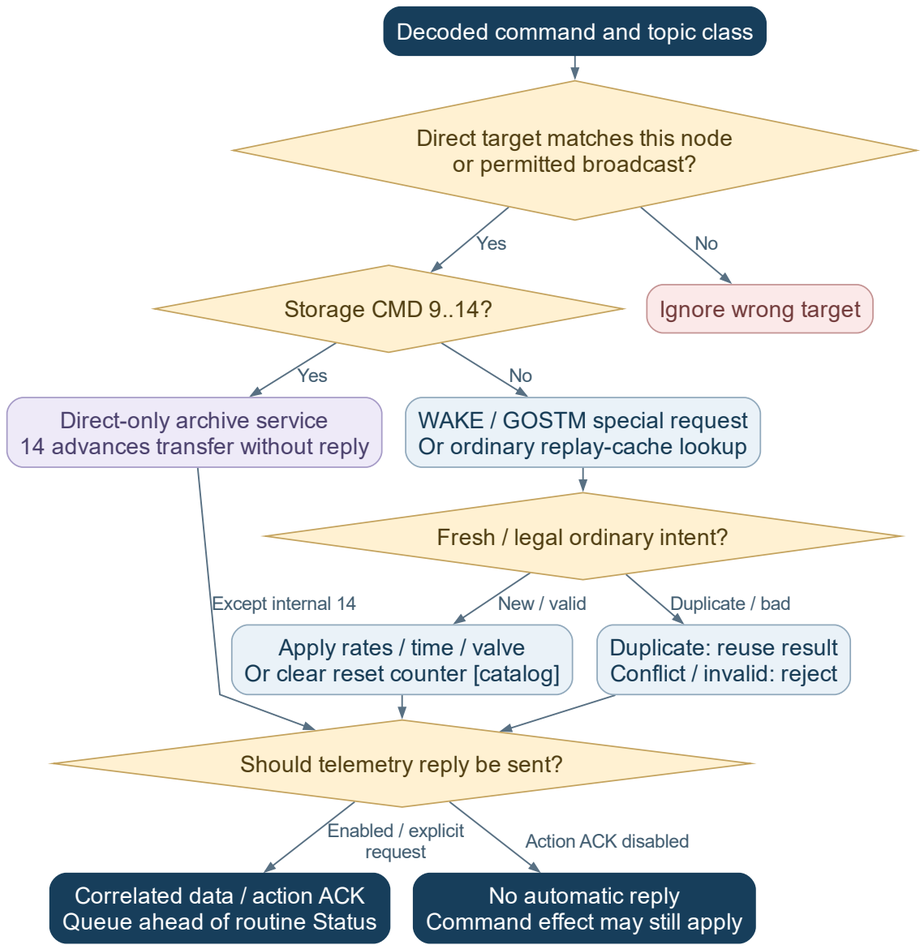
Source locations

Server/server_web.py:1648
Server/command_queue.py:180
Server/command_queue.py:305

Index >

Node command dispatch replay and replies

The shared command policy covers direct MQTT and LoRa. Internal archive ACKs and explicit data requests have distinct behavior.



- GOSTM plus FLASH_STATUS / LAST / TODAY explicitly request replies even when automatic action ACKs are off. CMD14 never creates a telemetry reply.

- Ordinary replay tracking is bounded RAM. Flash clear separately persists its most recent manual request ID; it is not an unlimited ledger of every past intent.

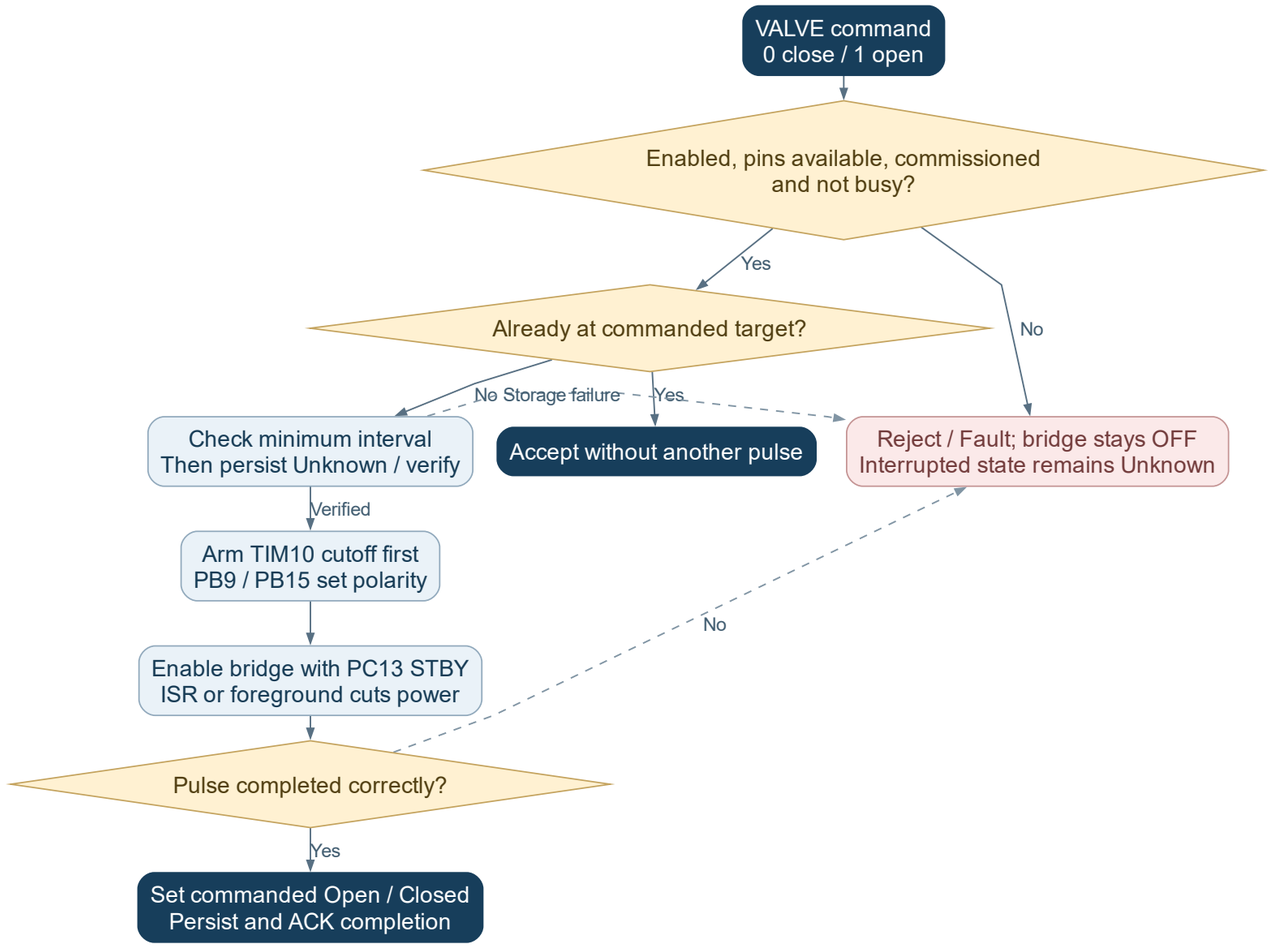
Source locations

Node/src/comm/CommandHandler.cpp:45
Node/src/comm/CommandHandler.h:35
Node/src/comm/CommandReplayCache.h:11

Index >

Commissioned latching valve pulse

The TB6612FNG drives a polarity pulse and then turns the bridge off. Telemetry records commanded state, not physical position feedback.



- LoRa uses PC13 for AUX, so the current profile inhibits the PC13 valve output. A new pin assignment needs an explicit wiring/configuration change.

- A completed pulse with a failed final durable save is still acknowledged as executed; RAM holds the target and reboot can recover Unknown. No WDT feeding is added inside the pulse wait.

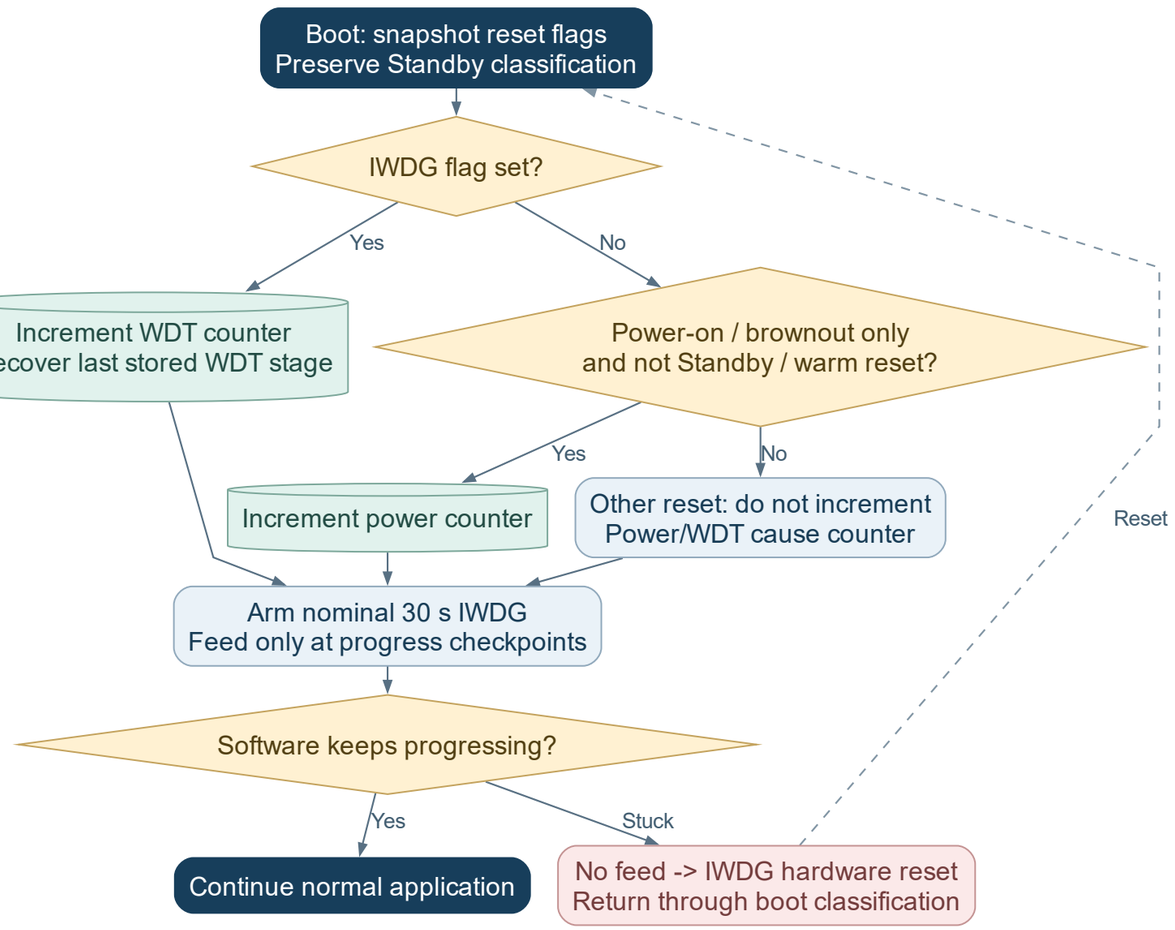
Source locations

Node/src/devices/ValveController.cpp:115
Node/src/devices/ValveController.cpp:98
Node/src/config/AppConfig.h:740

Index >

Watchdog reset causes and two counters

The hardware IWDG resets stalled active code. Only genuine power-on/brownout and IWDG causes increment their corresponding persistent counters.



- RESET-button / ST-Link software reset / planned Standby wake do not count as power resets. Deliberate WDT-based network recovery still counts as an IWDG reset.

- Direct CMD7 clears the power count; CMD8 clears the WDT count. Readback verification, duplicate checks and ACK result expose storage failure. RTC alarms, not the WDT, schedule normal wakeups.

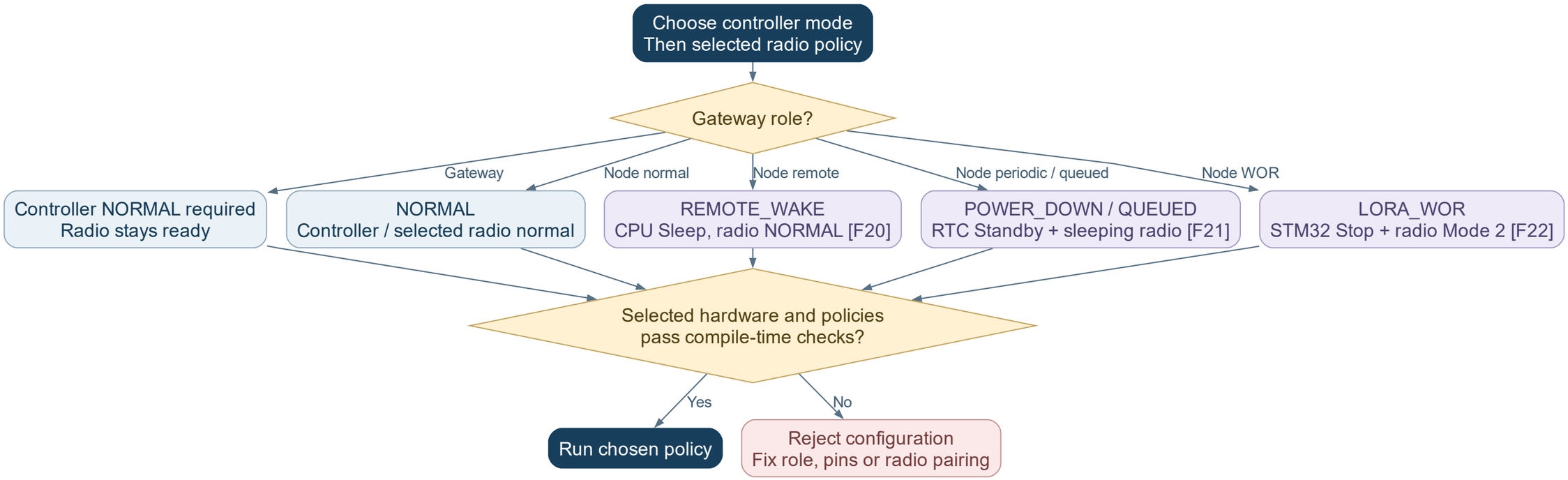
Source locations

Node/src/platform/Watchdog.cpp:126
Node/src/platform/ResetCause.h:11
Node/src/platform/WatchdogResetCounter.cpp:77

Index >

Select a supported power policy

Controller state and selected-radio policy must form a supported pair. The compiler rejects unsupported combinations.



- Direct SIM800 deep/queued Standby is rejected on this PCB: PWRKEY/DTR/RI are not routed. SIM may remain NORMAL while the controller uses REMOTE_WAKE.

- STM32 Stop reception is implemented through LoRa WOR Mode 2. This does not make an arbitrary normal modem able to wake the MCU from Standby. See the combination appendix.

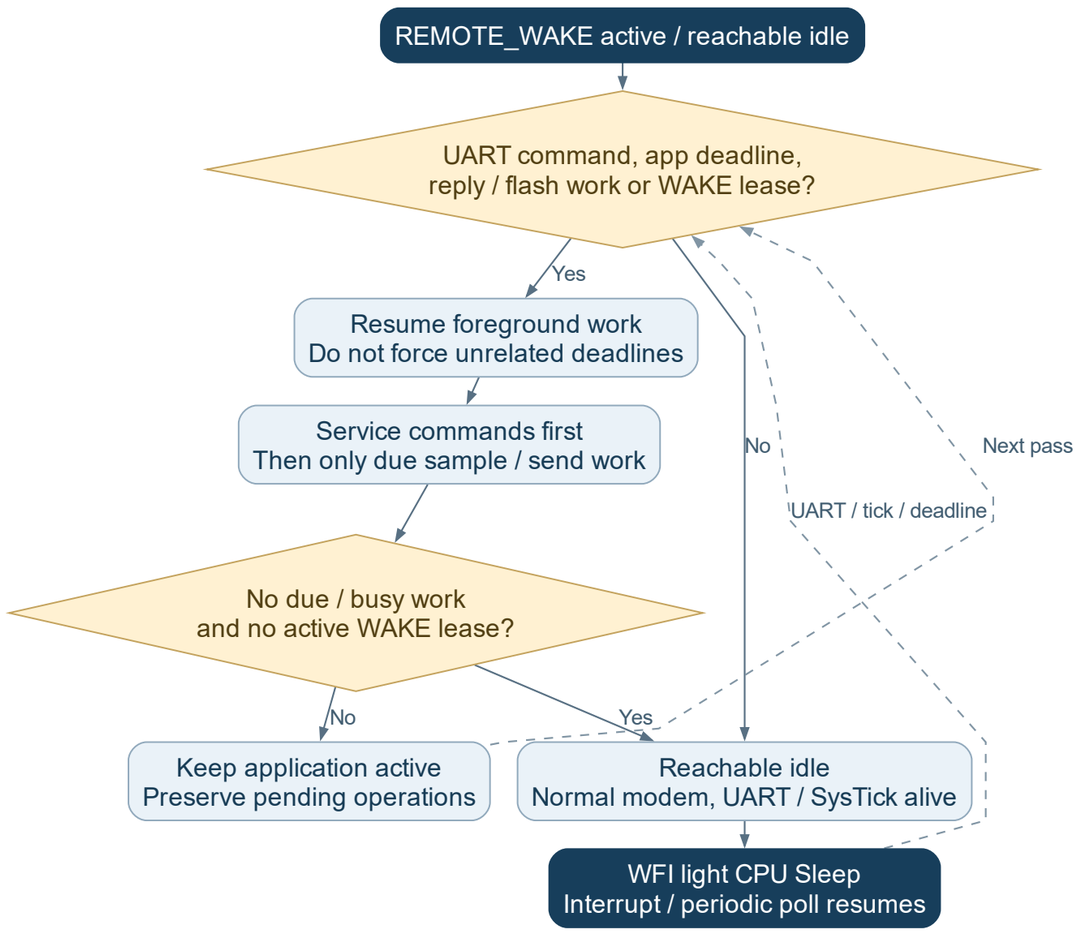
Source locations

Node/src/config/AppConfig.h:249
Node/src/power/PowerManager.cpp:1156

Index >

Remote wake with normal connected modem

The reachable-idle controller uses light CPU Sleep while the radio remains connected. The default policy responds to independent sampling and send deadlines.



- Default APP_REMOTE_WAKE_USE_APP_SCHEDULE=true uses SRATE and STRAN separately. false retains the older fixed-cycle remote policy.

- WAKE requests an active lease of 1..86400 s; data 0 uses the configured default. Flash work/transfer guards prevent inappropriate idling; this is not RTC Standby or a modem DTR sleep implementation.

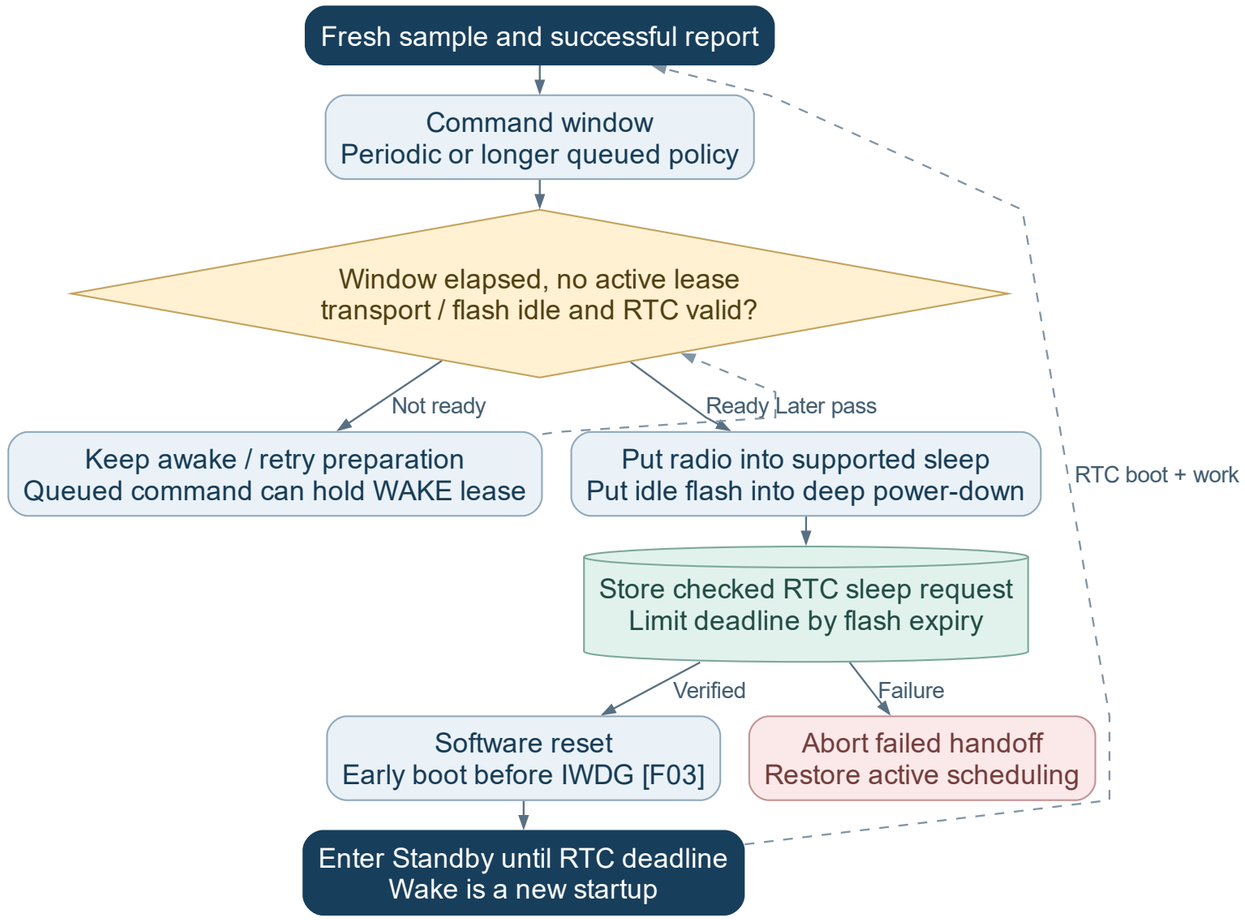
Source locations

Node/src/power/PowerManager.cpp:1314
Node/src/power/RemoteWakeCycle.h:72
Node/src/config/AppConfig.h:138

Index >

Periodic and queued RTC Standby

A successful report opens a command window. Standby is entered through a verified reset handoff so active work keeps watchdog protection.



- ESP8266 or LoRa periodic/queued profiles require the matching radio POWER_DOWN policy. The present direct SIM800 hardware rejects this mode.

- Queued server commands wait for the next Status; this does not make a fully sleeping modem instantly reachable. Flash write/recovery/transfer work defers Standby.

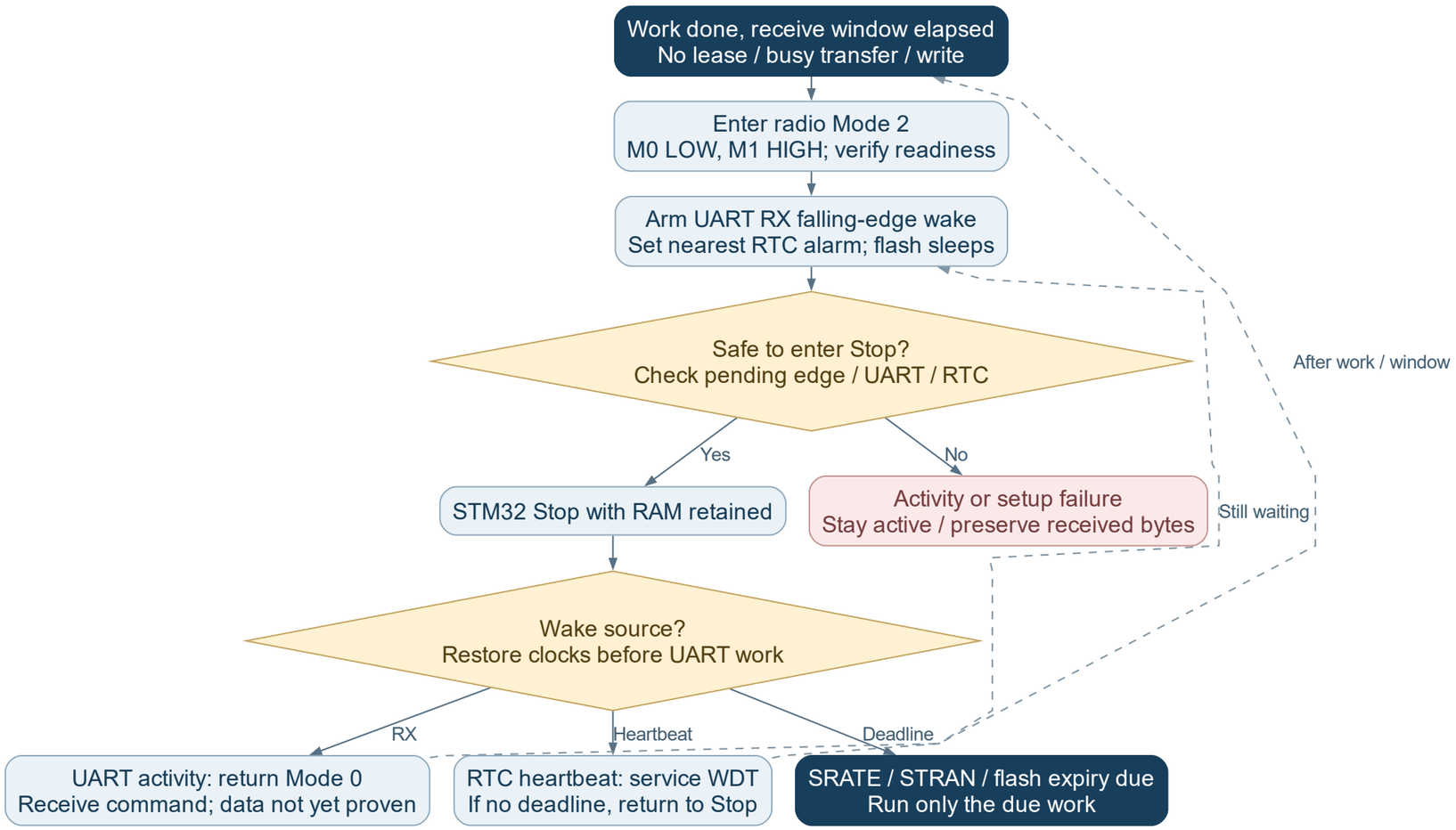
Source locations

Node/src/power/PowerManager.cpp:754
Node/src/power/PowerManager.cpp:858

Index >

LoRa wake on radio with STM32 Stop

The leaf listens in Mode 2 while STM32 retains RAM in Stop. UART RX wakes for radio traffic and RTC alarms schedule due work.



- Gateway WOR TX sends a Mode-1 probe before the command. AUX remains a readiness signal; the optional legacy AUX wake selector is separate from the default UART RX path.

- APP_LORA_WOR_USE_APP_SCHEDULE=true selects independent sampling and transmission deadlines. APP_POWER_SLEEP_SECONDS controls the legacy combined WOR cycle only when that switch is false. STIME must establish trusted RTC before Stop.

- The default 15 s RTC heartbeat services the nominal 30 s IWDG in foreground. It does not force a sensor read or routine Status. Stop residence ages the application deadlines using coherent RTC time because millis pauses. WDT expiry resets the controller.

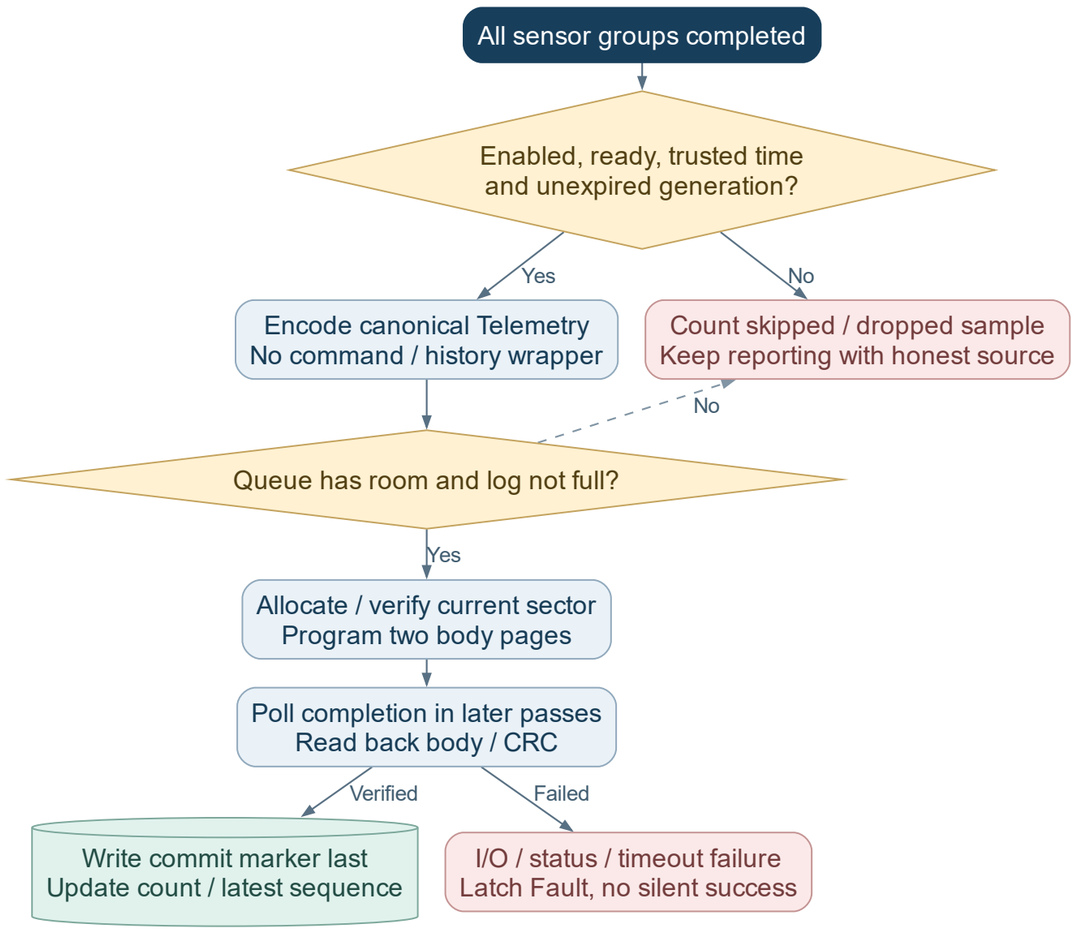
Source locations

Node/src/power/PowerManager.cpp:216
Node/src/power/WorWakeInput.h:30
Node/src/radio/LoraLink.cpp:1341

Index >

Capture and commit a flash sample

The external S25FL064L stores one canonical complete sample per accepted acquisition, with CRC and commit-last recovery.



- JEDEC 01 60 17 identifies the configured 8 MiB chip. Four RAM queue entries feed 512-byte slots with at most 472 canonical protobuf bytes each.

- Capacity is 14,322 records. Full means stop appending and count drops, not overwrite unexpired records. A return from append means queued, not yet durable.

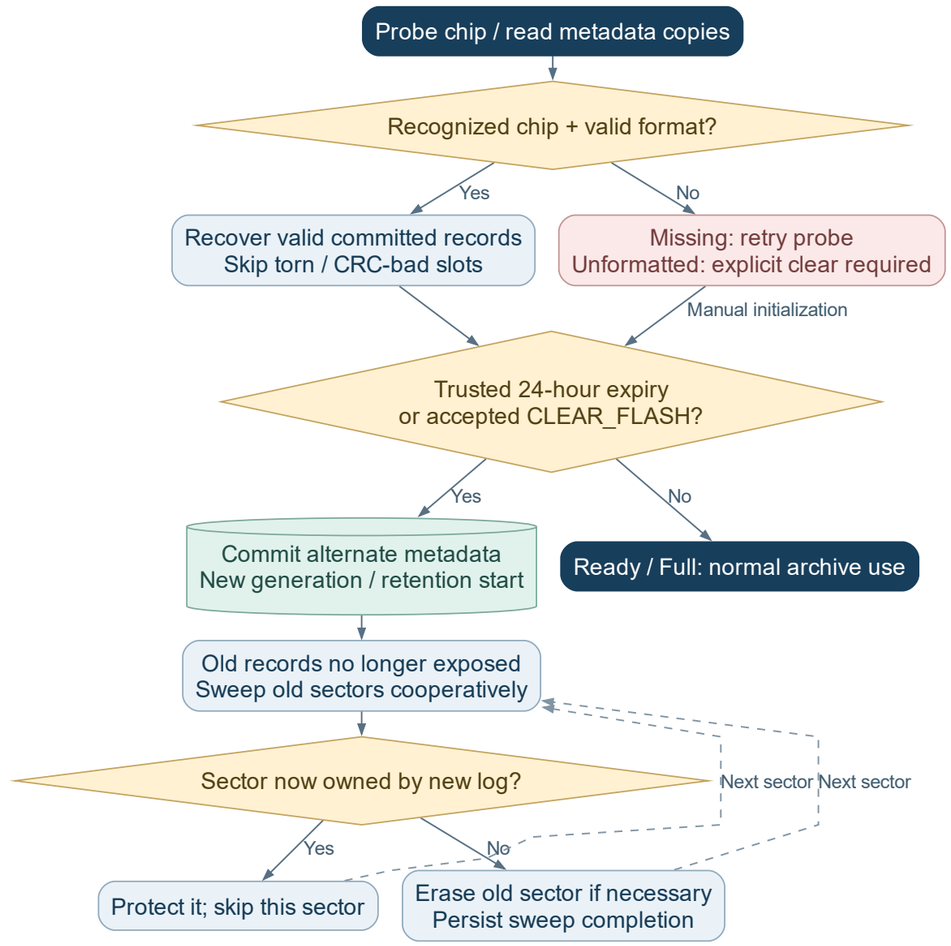
Source locations

Node/src/storage/TelemetryArchive.cpp:80
Node/src/storage/TelemetryJournal.cpp:100
Node/src/storage/S25Fl064l.cpp:40

Index >

Flash recovery retention and clearing

Two metadata copies choose a durable generation. Records recover cooperatively; retention changes the generation before background physical reclamation.



- Retention is 24 elapsed hours from the persisted batch start, not local midnight. Untrusted/backward time cannot trigger automatic erase.

- Ownership is rechecked on every sweep pass so newly allocated samples cannot be erased. Clear ACK is acceptance; physical erase progress may continue across active windows.

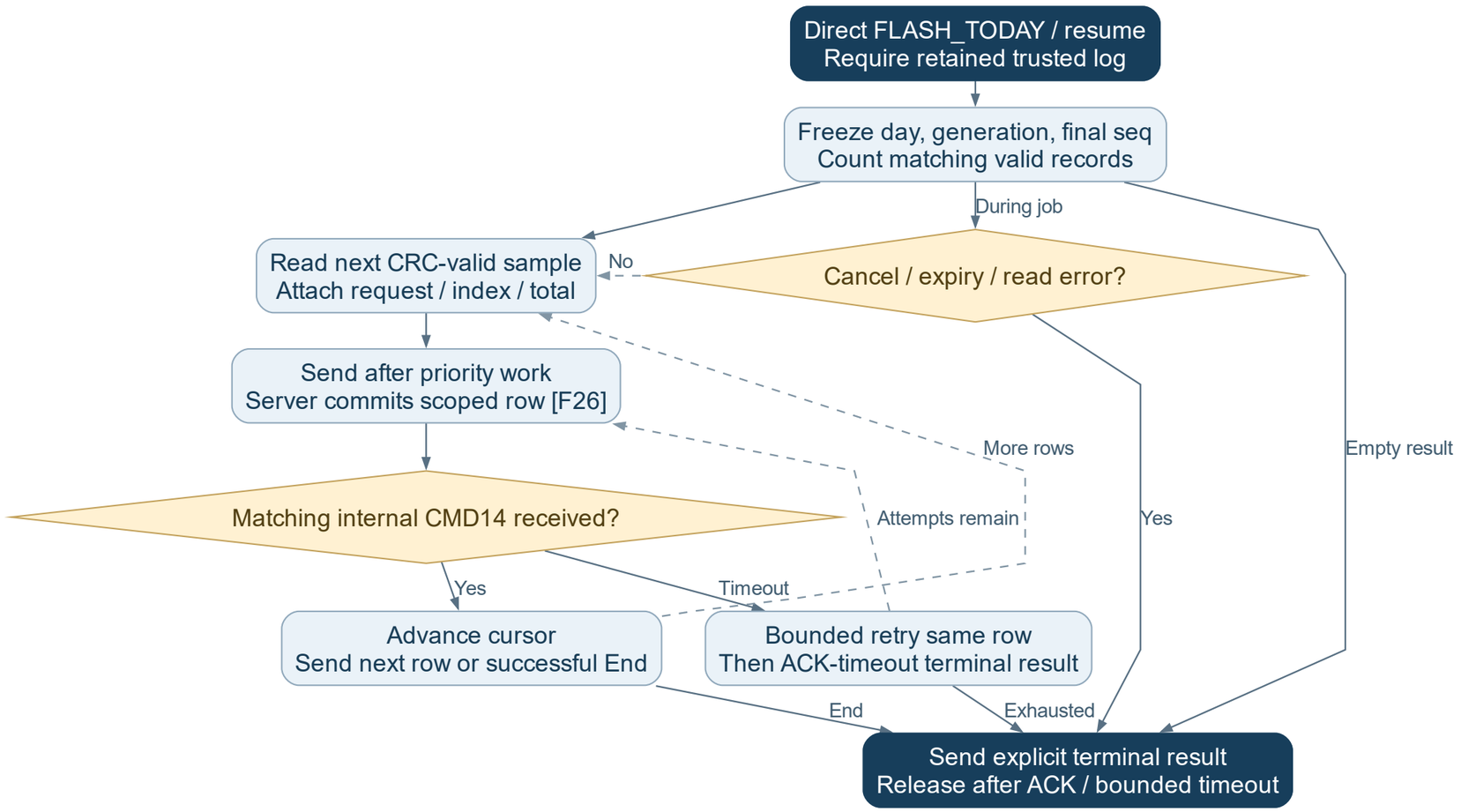
Source locations

Node/src/storage/TelemetryJournal.cpp:26
Node/src/storage/TelemetryJournal.cpp:159
Node/src/storage/TelemetryJournal.cpp:78

Index >

Download retained history with persistence ACK

FLASH_TODAY freezes the node day and final sequence. The node sends one indexed row at a time and advances only after server storage acknowledgment.



- FLASH_LAST is a single original sample, not a new sensor poll. TODAY covers only the retained batch portion of the current node calendar day, not necessarily a complete day.

- New acquisition remains allowed while download keeps the controller awake; later records are excluded from the frozen job. Server history does not feed live charts or consumption.

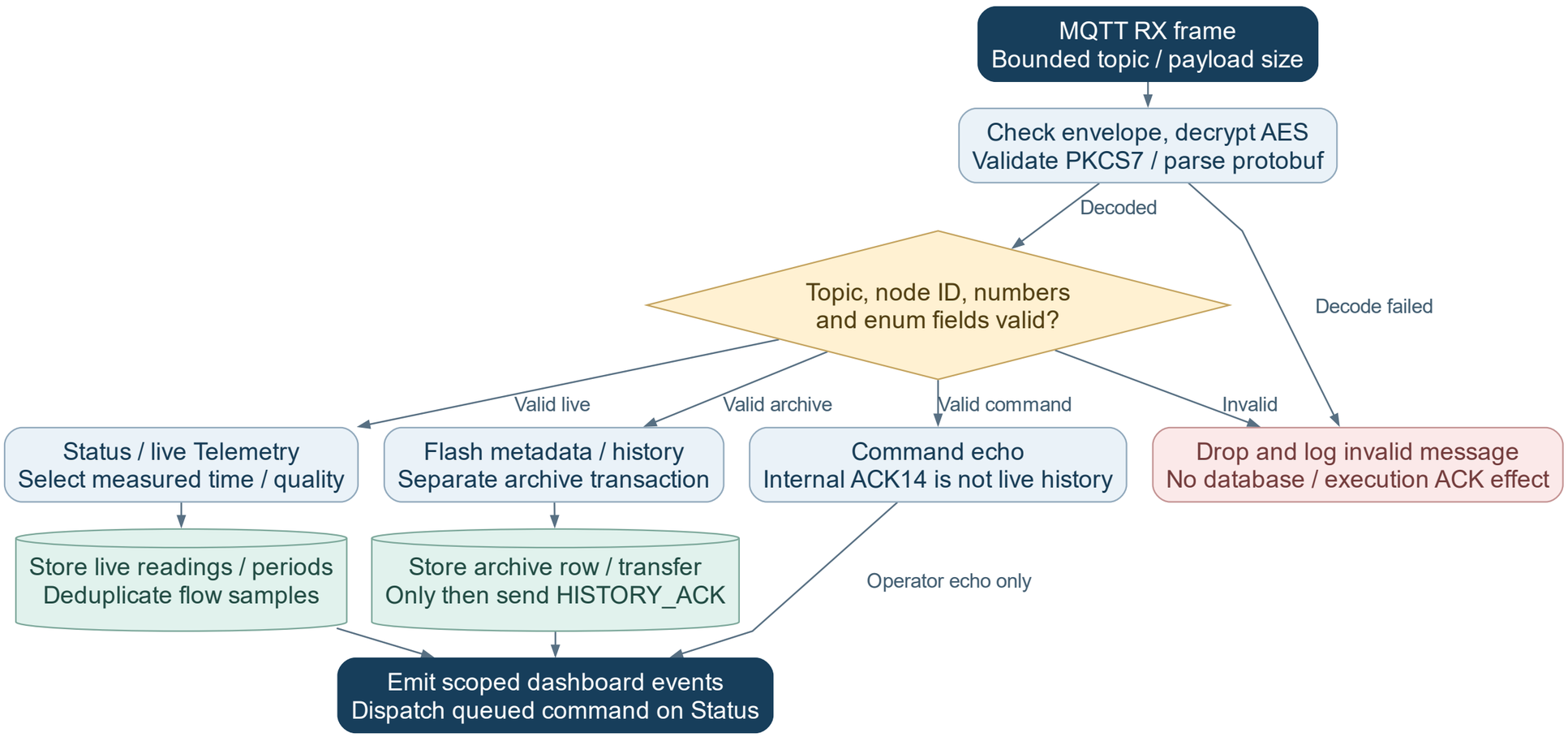
Source locations

Node/src/storage/TelemetryArchive.cpp:201
Server/flash_archive.py:74
Server/server_web.py:134

Index >

Server receive validate and store

Incoming MQTT frames are decrypted and checked against exact regional topic identity before any live update, command receipt or archive persistence.



- Topics are wiot/company/region/node/status, /data, /cmd, plus regional /broadcast. Topic node and protobuf node must agree; a good AES decode alone is insufficient.

- Stored sample_time_valid is independent of the primary meter RTC flag. Original acquisition time is used when trusted, with explicit fallback provenance in live history.

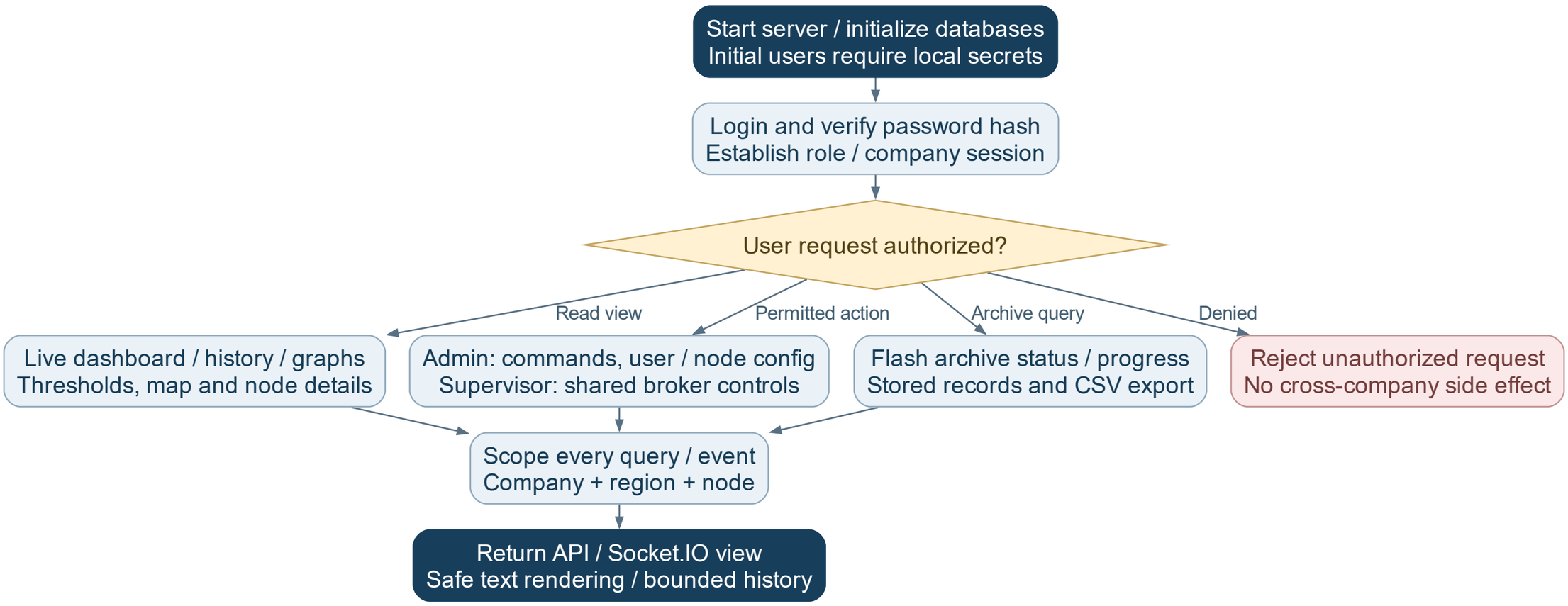
Source locations

Server/mqtt_impl.py:67
Server/server_web.py:987
Server/input_validation.py:50

Index >

Server users dashboard reports and isolation

The web application separates authenticated views, company scope and privileged infrastructure actions while displaying live and archived data.



- Live readings/periods, command intentions/receipts and external-flash downloads have different persistence paths. Clearing a node log does not delete already received server archive rows.

- Broker connection readiness includes subscription success. Ordinary company users do not receive shared infrastructure logs or control another company through a changed browser selector.

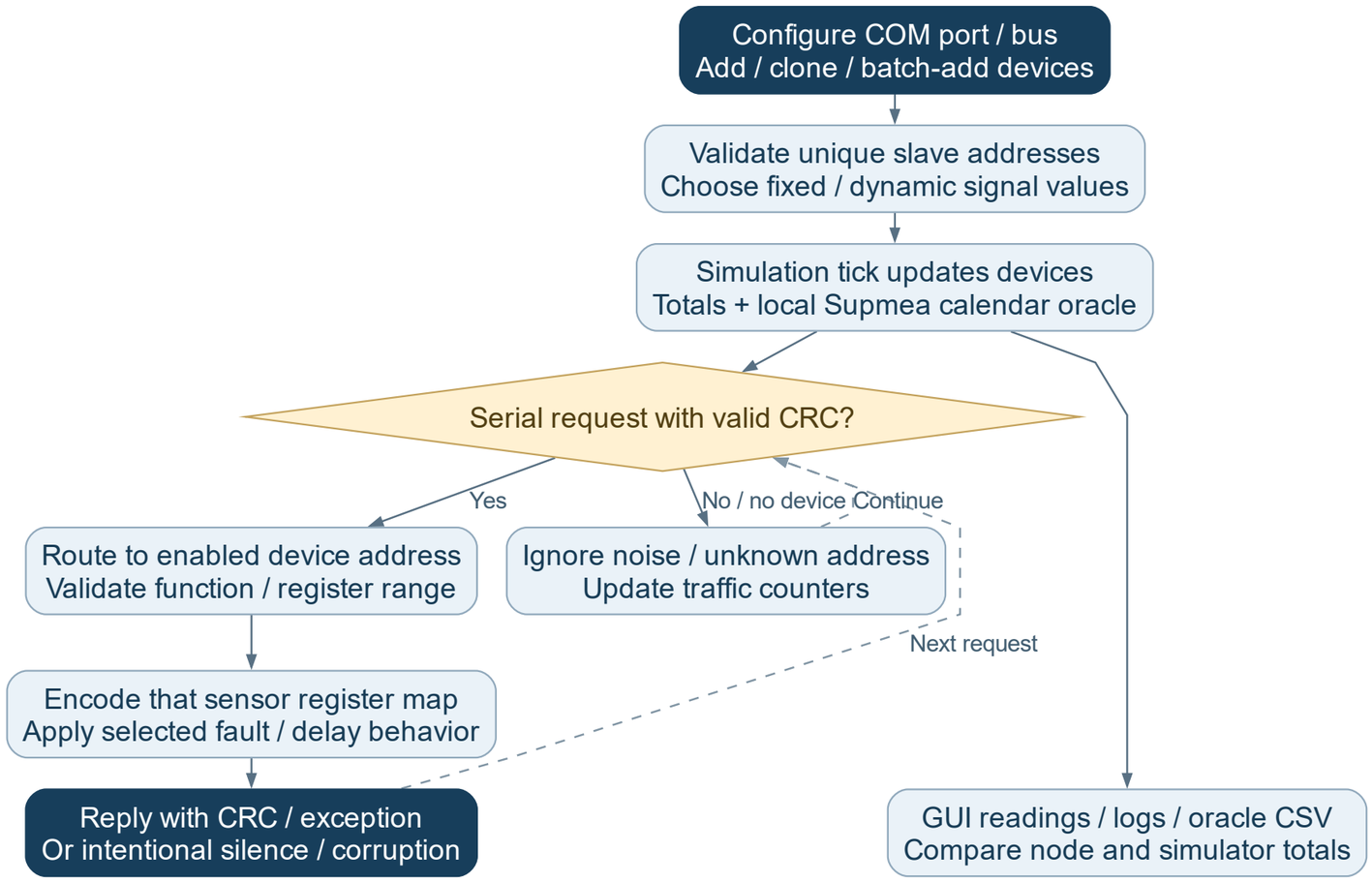
Source locations

Server/server_web.py:1211
Server/server_web.py:1205
Server/static/js/flash_archive.js:32

Index >

Four sensor simulator on one shared bus

The multi-sensor lab hosts multiple virtual devices per sensor type, each with a unique address and independently configured signal behavior.



- Several HNDA, TUF, Supmea and Sondas devices can share the simulator bus. The node still polls only its configured addresses; duplicate addresses on a physical bus are invalid.

- The independent Supmea day/month/year reference stays local to the simulator and is not injected into the RS485 meter map. Focused TUF/Supmea simulators remain available.

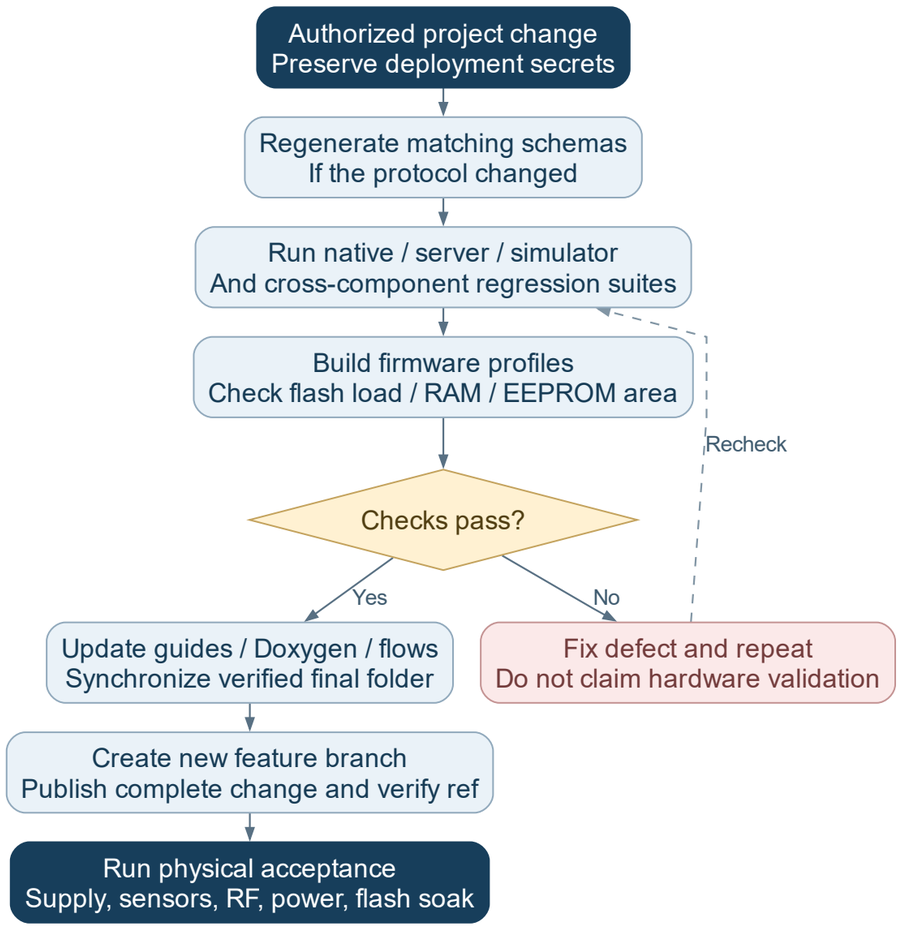
Source locations

MultiSensor/multi_sensor_simulator.py:606
MultiSensor/multi_sensor_bus.py:51
MultiSensor/multi_sensor_core.py:866

Index >

Verify document and release the project

A release includes matched node/server schemas, behavioral regressions, firmware bounds, human documentation and a separate hardware acceptance step.



- Internal firmware flash load is kept below the reserved EEPROM sector at 0x08020000. Source/docs delivery does not flash the board or prove physical operation.

- Run Validate-Release.ps1 with installed tools. Commissioned bench values require the existing -UseFactoryConfigurationFixture for valve guard tests. This fixture does not edit the deployment header.

- Keep .env, AppSecrets, private AppConfig.cpp and runtime databases out of Git. Separate firmware ports have their own release history.

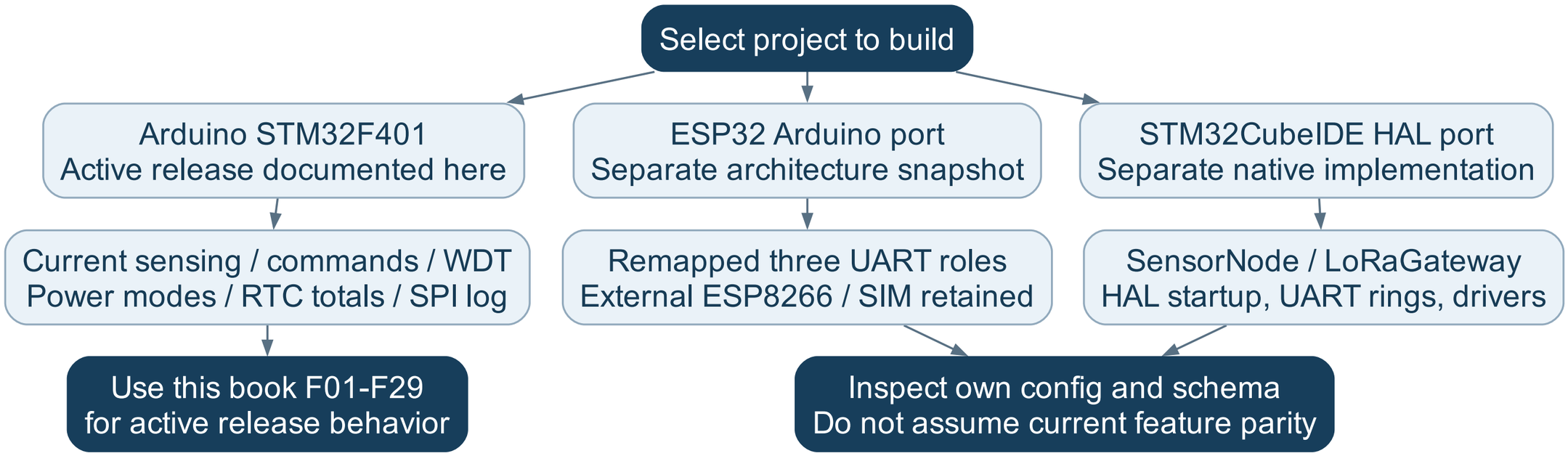
Source locations

Validate-Release.ps1:14
regenerate_protocol.py:15
Node/SPI_FLASH_TEST_PLAN.md:1

Index >

Active release and separate firmware ports

The repository contains three firmware implementations. The detailed current flowcharts describe the Arduino STM32 release; alternate ports retain their own snapshots.



- The ESP32 compatibility port uses external modems; it does not silently switch the project to ESP32 internal WiFi. Its UART mapping is role dependent.

- Neither alternate port has been updated by the recent active STM32 storage/power/command releases. Historical compatibility claims must be checked against that port's actual schema and code.

Source locations

ESP32/ESP32_USER_GUIDE.md:3
CubeIDE/README.md:1
Node/README.md:1

Index >

Command catalog

Use matching topic identity and a server-generated request ID. Explicit data requests and internal protocol ACKs differ from optional automatic action replies.

ID	Name	Data	Scope	Behavior
1	SRATE	10..86400 seconds	Ordinary / scoped	Set the node sampling period; node bounds are authoritative.
2	STRAN	10..86400 seconds	Ordinary / scoped	Set the node routine transmission period.
3	STIME	Server UTC epoch	Ordinary / scoped	Validate and synchronize RTC; establish trusted time.
4	GOSTM	No useful data	Data request	Return current runtime Telemetry; explicit reply remains enabled.
5	VALVE	0 close; 1 open	Ordinary / scoped	Commissioned polarity pulse; state is commanded, not measured.
6	WAKE	0 default; 1..86400 s	Supported power modes	Hold an active lease in REMOTE_WAKE, RTC_QUEUED or LORA_WOR.
7	CLEAR_POWER_RESETS	0 + nonzero request ID	Direct node only	Persist and verify power counter zero; MCU is not reset.
8	CLEAR_WDT_RESETS	0 + nonzero request ID	Direct node only	Persist and verify IWDG counter zero; MCU is not reset.
9	CLEAR_FLASH	0 + nonzero request ID	Direct; trusted RTC	Accept a new log generation; physical reclamation is asynchronous.
10	FLASH_STATUS	0	Direct data request	Return state, capacity, used/free slots, frame count and progress.
11	FLASH_LAST	0	Direct data request	Return last retained committed sample with original acquisition time.
12	FLASH_TODAY	0 or packed resume cursor	Direct data request	Freeze day/generation/final sequence and stream retained rows.
13	FLASH_CANCEL	0	Direct node only	Cancel active export; does not erase the stored log.
14	HISTORY_ACK	(phase << 32) | index	Internal; same job ID	Confirm server persistence; advance the transfer without an ACK loop.

SRATE/STRAN are seconds in CMD data and milliseconds in Telemetry. The active node minimum is 10 s; broader server API input can still be rejected by the node. Automatic action replies follow APP_ENABLE_COMMAND_ACK_REPLIES; GOSTM and explicit storage queries still respond.

Supported power combinations

Values refer to controller and selected-radio macros in the active Arduino STM32 project. Policies for physically absent, unselected modules do not create independent power switches.

Role	Transport	Controller selector	Active radio selector	Wake or work source	Behavior / constraint
Sensor node	WiFi / SIM / LoRa	NORMAL (0)	NORMAL (0)	Continuous active operation	No automatic controller deep sleep.
Sensor node	WiFi / SIM / LoRa	REMOTE_WAKE (2)	NORMAL (0)	UART/server command + app schedule	Light CPU Sleep; modem remains connected.
Sensor node	ESP8266	POWER_DOWN (1)	POWER_DOWN (1)	RTC report-cycle deadline	ESP deep sleep/reset path; STM32 Standby.
Sensor node	LoRa	POWER_DOWN (1)	POWER_DOWN (1)	RTC report-cycle deadline	Radio sleeps; no immediate downlink during Standby.
Sensor node	ESP8266	QUEUED (4)	POWER_DOWN (1)	RTC + queued command after Status	Longer command window, optional active lease.
Sensor node	LoRa	QUEUED (4)	POWER_DOWN (1)	RTC + queued command after Status	Gateway queues do not instantly wake a fully sleeping radio.
Sensor node	LoRa	LORA_WOR (3)	WAKE_ON_RADIO (2)	UART RX / RTC heartbeat or deadline	STM32 Stop; E32 Mode 2 receive policy.
Gateway	WiFi / SIM uplink	NORMAL (0)	Selected modem NORMAL	Always services LoRa and MQTT	LoRa is normal except temporary Mode-1 command wake TX.
Rejected	Direct SIM800	POWER_DOWN / QUEUED	Any deep-sleep selection	Not supported on current PCB	Required PWRKEY/DTR/RI routing is absent.
Rejected	Any selected modem	REMOTE_WAKE	POWER_DOWN / WOR radio	Cannot receive as this policy expects	Use NORMAL selected radio with reachable CPU Sleep.
Rejected	Gateway	Any non-NORMAL	Any	Gateway service must stay available	Build-time guard rejects the combination.

Power protocol telemetry uses 1 NORMAL, 2 RTC_PERIODIC, 3 REMOTE_WAKE, 4 RTC_QUEUED and 5 LORA_WOR_STOP, which are not the same numbers as the compile-time controller selectors. Standby restarts startup; light Sleep and Stop retain their respective runtime state.